Aspen

A security protocol notation

Anders Andersen
UiT The Arctic University of Norway

2026/05/12 10:02:53
(aspen.sty version 2.5)

In security literature, different notations for cryptographic values, functions and protocols have been used and suggested. Three often cited references for such notations are "Kerberos: An Authentication Service for Open Network Systems" [23], "Exploring Kerberos, the Protocol for Distributed Security in Windows 2000" [13], and "A Formal Semantics for Protocol Narrations" [10]. The notation Aspen presented here is strongly inspired by notations found in these three references, notations found in text books [7], and the notation used in my own publications, teaching and presentations. Aspen is closely related to what is often called "security protocol notation", "standard protocol engineering notation" [5, 6], "standard protocol notation" [7], or "protocol narrations" [10].

This text documents the Aspen[1] notation and how this notation can be used in LaTeX documents using the LaTeX package aspen [2]. Since the LaTeX package aspen optionally provides support for the BAN logic notation, the BAN logic notation is included in the documentation.

The Aspen notation is *not* a formalism, like BAN (Burrows–Abadi–Needham) logic [12], or a calculus for analysis of cryptographic protocols, like Spi calculus [1]. For a more detailed analysis of cryptographic protocols, more expressive notations like BAN logic, Spi calculus, or something similar should be considered. Other references presenting relevant notations include, but are not limited to: [11, 14, 22].

$$
\begin{aligned}
&A \longrightarrow S : \{A, B, N'_A\} \\
&S \longrightarrow A : \{N'_A, B, K_{A,B}, \{K_{A,B}, A\}_{K_{B,S}}\}_{K_{A,S}} \\
&A \longrightarrow B : \{K_{A,B}, A\}_{K_{B,S}} \\
&B \longrightarrow A : \{N'_B\}_{K_{A,B}} \\
&A \longrightarrow B : \{N'_B - 1\}_{K_{A,B}}
\end{aligned}
$$

Figure 1: Aspen example (what protocol is it?)

Contents

1 Introduction 2

2 The notation 3
2.1 Aspen 3
2.2 BAN logic 8

3 Use the notation in text 10
3.1 Aspen 11
3.2 BAN logic 22
3.3 Series of steps 23

4 Notation usage examples 26

A References 35

B Notes 37
B.1 Notes on the notation 37
B.2 Notes on the typesetting options 38
B.3 Notation example listing 40

[1] Originally, I had no intention to name the notation presented here. While working on this text, it became clear that it was inconvenient to not be able to refer to the notation with a short name. The name Aspen *can* be an abbreviation for "A Security Protocol Engineering Notation", but for me it is now short for "Anderson-inspired Standard Protocol Engineering Notation", in memory of the late Professor Ross J. Anderson who has meant so much for the fields of computer security, distributed systems, and, in particular, security engineering [5, 6, 7].

Values:	$true$, $\{m\}$, $H\{m\}$	Values, structured values and typed structured values. In this notation a message is seen as a structured value.
Principals:	A, B, S	Principals in security protocols, including clients, servers, and other participants.
Keys:	$K_{A,B}$, $K'_{C,S}$, K_A^+, K_B^-	Cryptographic keys, used to encrypt, decrypt, sign and verify values and messages.
Nonces:	N'_A, N'_B, N'_S	Nonces are generated to be fresh and commonly include a timestamp or a number that is used only once.
Counter:	I_A, C_B	Counter or indexes can be used to identify a session or a number in a sequence.
Random:	R_x, R'_1	Random values can be a variant of nonces. The $'$ mark hints about limited useful lifetime (once, or during a session).
Time:	T_S, T_A, T_L	Timestamps and lifetime are often used, together with nonces, to avoid replay and session keys that are too old.
Strings:	"Hello world!"	Not necessary for the intended notation usage, but text strings are often found in examples in the literature.
Variables:	x, y, z, a, b, c	A variable can be assigned a value. Might also be used in the context of "we are not sure about its value".
Functions:	$H(m)$, $Func(x,y) \rightarrow z$	A function can take arguments and produce a value. Some functions are the constructor of typed structured values.
Procedures:	$Abort()$	A procedure is similar to a function but doesn't produce a value. Instead is has an affect on the running protocol (e.g. failing).
Labels:	M_1, S_1	Labels are used to label steps when a security protocols is presented as a series of steps.
LaTeX code:	`\AFunc{Func}{x,y}`	LaTeX code is shown when documenting the usage of the notation in text using the LaTeX package `aspen`.

Figure 2: In the text, color is used to distinguish different features.

1 Introduction

Why Aspen then? One motivation is to have an expressive notation that can be used in publications where security protocols are presented. Another motivation is to have a notation that can be used when teaching security related topics. This text is an attempt to document a notation that have been used and refined over years. The notation should be familiar, but with some new useful refinements and contributions not found in similar notations. It should also be possible to use the notation together with other notations, like BAN logic. A more detailed discussion on the choices made for the Aspen notation is found in Appendix B.1, *Notes on the notation*. A short introduction to Aspen is published in TUGboat [2]. The LaTeX package can be downloaded from CTAN or its home:

`https://www.pg12.org/dist/texmf/tex/latex/aspen/`

When using Aspen, colors can be used to distinguish different types of features. Figure 2 illustrates how the different features are colored. Colors are optional when using the notation. They are enabled by the `color` option to the LaTeX package `aspen`. Colors are only added for readability. The LaTeX package `aspen` provides different color profiles for typesetting the notation (see Appendix B.2, *Notes on the typesetting options*).

If the `aspen` package is loaded with the option `ban`, the BAN logic notation from "A Logic of Authentication» [12] is included (see Section 2.2 and 3.2).

2 The notation

In the description of the notation below, notation that might be obvious is included. It is done for completeness and consistency. For some notation constructs, usage examples are provided. These examples might include notation constructs explained later in the text.

2.1 Aspen

Notation	Description
$=, <, \leq, >, \geq$	$=$ means *"is equal"*, either as a statement or a claim (e.g., a claim that can be, or has to be, verified). $<$, $\leq$, $>$ and $\geq$ means *"less than"*, *"less than or equal"*, *"greather than"*, and *"greather than or equal"*, respectively. These notation constructs are typically used to compare counters and timestamps in protocols.
$\oplus$, .	The binary operator $\oplus$ is exclusive or, and the binary operator . is concatenation (used to concatenate two values or strings). The concatenation operator has precedence over the exclusive or operator. In Section B.2, other options for the binary concatenation operator is presented.
$\Rightarrow, \Leftrightarrow$	$x \Rightarrow y$ means *"y, if x"*. $x \Leftrightarrow y$ means *"y, if and only if x"*. This is an example used with the *Verify* function (see below for the description of other parts of the notation used in the example): $$Verify\left(K_A^+, \{m\}^{K_x}\right) = true \iff K_x = K_A^-$$
$v \leftarrow 0, v \leftarrow v + 1$	The left arrow is used for assignments. It is sometimes useful to give names to values and be able to change named values.
true, *false*	The boolean values *true* and *false*. The value *true* will also be used to show that an operation completed with success (if that is important). For example, when we verify a digital signature and it is found valid, the function doing the verification returns the value *true*. Otherwise, the value *false* is returned.
e **?** $c_1 : s_1$ **;;** ...	Conditionals can be used to change the behavior of a protocol based on a condition. A conditional consists of an optional expression followed by question mark (**?**) followed by conditional steps. Each conditional step consist of a condition followed by a colon (**:**) followed by the step that should be performed if the condition is true. Conditional steps are separated by a single or a double semicolon (**;** or **;;**). Conditional step s_1 is performed if condition c_1 is fulfilled, and so on. Fulfilled means that the condition match the expression, or if no expression is given, the condition is true. The two semicolons **;;** mark a break, meaning that if the condition before the two semicolons is *true* and the conditional step is performed, the rest of the conditional steps are ignored (and not evaluated and performed). If a single semicolon **;** is used, the behavior is different; even if the conditional step before the semicolon is performed, the following conditional steps are evaluated and will be performed if their conditions are fulfilled. The following two examples are semantically the same; they assign the value $N'_y + 1$ to v if V is equal to $N'_y + 1$ and otherwise abort the protocol: $$V = N'_y + 1 \text{ ? } true : v \leftarrow v + 1 \text{ ;; } false : \underline{Abort}()$$ $$\text{? } V = N'_y + 1 : v \leftarrow v + 1 \text{ ;; } V \neq N'_y + 1 : \underline{Abort}()$$

$\wedge, \vee, \neg$	Logical *and*, *or* and *not*, respectively. Can be used in the conditions of conditional steps.
$x \rightsquigarrow y$	We use an unpack (leads-to) arrow $\rightsquigarrow$ to show more details or to unpack a value or a message. The following example shows that a digital signature is actually a cryptographic hash value of the message encrypted with a private key (see below for the description of other parts of the notation used in the example): $Sig\{m\}^{K_A^-} \rightsquigarrow \{H\{m\}\}_{K_A^-}$
K_X	A shared or secret key, also known as a symmetric encryption and decryption key.
$K_{A,B}$	A shared key for A and B (this notation can be extended, for example with $K_{A,B,C}$ for a key shared between A, B, and C, or $K_{M_1-M_n}$ for a key shared among n group members $M_1, \ldots, M_n$).
$K'_{C,S}$	A session key for C and S (this notation can be extended, for example with $K'_{A,B,S}$ for a key session key shared between A, B, and S, or $K'_{M_1-M_n}$ for a session key shared among n group members $M_1, \ldots, M_n$).
K_A^+	The public key of a public-private key pair of A: (K_A^+, K_A^-).
K_A^-	The private key of a public-private key pair of A: (K_A^+, K_A^-).
K_P''	A secret key generated from the password P.
$\{m\}$	A structured value or message containing m. A structured value can be nested.
$T\{m\}$	A structured value or message with the type T. An example of a structured value marked with the type *Sig*: $Sig\{m\}^{[A]}$ has the type *Sig* and the superscript $[A]$ (signed by A)
$A \longrightarrow B : \{m\}$	Message $\{m\}$ sent from A to B.
$Func(x, y)$	A function *Func* with two arguments x and y (*Encrypt* and *Decrypt* described below are examples of such functions).
$Func(x, y) \rightarrow z$	We use an arrow $\rightarrow$ to illustrate what a function produces (in this case z). The following example shows that the function *Encrypt* produces a ciphertext encrypted with the given shared key (see below for the description of other parts of the notation used in the example): $Encrypt(K_{A,B}, m) \rightarrow \{m\}_{K_{A,B}}$
Proc(“text”)	A procedure *Proc* with an optional comment “text” Procedures are similar to functions, but they don’t produce a value. Instead they have an affect on the running protocol, and they are typically used if something fails or some conditions are not satisfied. The name of procedures are underlined.
Abort()	The procedure *Abort* immediately aborts the current protocol. An optional comment can be given: *Abort*(“Comment”)

$\{m\}_{\square}$	In general, a subscripted structured value means an encrypted value or message (a ciphertext), where the subscript $\square$ represents the encryption key (or the holder of the encryption key). This is an example where the plain text m is encrypted with the encryption key K: $\{m\}_K$
$\{m\}^{\square}$	In general, a superscripted structured value means a signed value or message, where the superscript $\square$ represents the key used to sign (or the holder of the key used to sign). This is an example where the plain text m is signed by principal A: $\{m\}^{[A]}$
$Encrypt(K_{A,B}, m)$	Encrypt plain text m with shared key $K_{A,B}$: $Encrypt(K_{A,B}, m) \rightarrow \{m\}_{K_{A,B}}$
$Decrypt(K_{A,B}, c)$	Decrypt cipher text c with shared key $K_{A,B}$, where $c = \{m\}_{K_{A,B}}$: $Decrypt(K_{A,B}, c) \rightsquigarrow Decrypt\left(K_{A,B}, \{m\}_{K_{A,B}}\right) \rightarrow m$
$Encrypt(K_A^+, m)$	Encrypt plain text m with public key K_A^+: $Encrypt(K_A^+, m) \rightarrow \{m\}_{K_A^+}$
$Decrypt(K_A^-, c)$	Decrypt cipher text c with private key K_A^-, where $c = \{m\}_{K_A^+}$: $Decrypt(K_A^-, c) \rightsquigarrow Decrypt\left(K_A^-, \{m\}_{K_A^+}\right) \rightarrow m$
$H\{m\}$	A cryptographic hash value of m.
$H(m)$	A cryptographic hash function producing the cryptographic hash value of m: $H(m) \rightarrow H\{m\}$
$MAC\{m\}^{K_A}$	The message authentication code of m with key K_A.
$CMAC\{m\}^{K_A}$	The cipher-based message authentication code of m with key K_A.
$HMAC\{m\}^{K_A}$	The HMAC message authentication code [9] of m with key K_A.
$MAC(K_A, m)$	The message authentication code function producing the message authentication code of m with key K_A: $MAC(K_A, m) \rightarrow MAC\{m\}^{K_A}$
$CMAC(K_A, m)$	The cipher-based message authentication code function producing the cipher-based message authentication code of m with key K_A: $CMAC(K_A, m) \rightarrow CMAC\{m\}^{K_A}$

$HMAC(K_A, m)$	The HMAC message authentication code function producing the HMAC message authentication code of m with key K_A: $$HMAC(K_A, m) \rightarrow HMAC\{m\}^{K_A} \rightsquigarrow H\left\{(\overline{K}_A \oplus opad) \,.\, H\left\{(\overline{K}_A \oplus ipad) \,.\, m\right\}\right\}$$ $$\overline{K}_A = \begin{cases} H\{K_A\} & \text{if } K_A \text{ is larger than block size} \\ K_A & \text{otherwise} \end{cases}$$ The two block-sized paddings, $opad$ (outer padding) and $ipad$ (inner padding), each consists of a repeating byte value (0x5c and 0x36, respectively).
$Sig\{m\}^{[A]}$	Digital (cryptographic) signature of m signed by A.
$Sig\{m\}^{K_A^-}$	Digital (cryptographic) signature of m signed with private key K_A^-: $$Sig\{m\}^{K_A^-} \rightsquigarrow \left\{H\{m\}\right\}_{K_A^-}$$
$Sig\{m\}^{[A,B]}$	Digital (cryptographic) signature of m based on a shared secret between A and B.
$Sig\{m\}^{K_{A,B}}$	Digital (cryptographic) signature of m signed with the shared key $K_{A,B}$ (a shared secret between A and B): $$Sig\{m\}^{K_A} \rightsquigarrow \left\{H\{m\}\right\}_{K_{A,B}}$$
$Sig(K_A^-, m)$	Function creating a digital (cryptographic) signature of m with private key K_A^-: $$Sig(K_A^-, m) \rightarrow Sig\{m\}^{K_A^-}$$
$Sig([A, B], m)$	Function creating a digital (cryptographic) signature of m based on a shared secret between A and B: $$Sig([A, B], m) \rightarrow Sig\{m\}^{[A,B]}$$
$Sig(K_{A,B}, m)$	Function creating a digital (cryptographic) signature of m with the shared key $K_{A,B}$ (a shared secret between A and B): $$Sig(K_{A,B}, m) \rightarrow Sig\{m\}^{K_{A,B}}$$
$\{m\}^{[A]}$	m is signed by A (m signed is a combination of m itself and a digital signature of m, in this case a digital signature signed by A): $$\{m\}^{[A]} \rightsquigarrow \left\{m, Sig\{m\}^{[A]}\right\}$$
$\{m\}^{K_A^-}$	m is signed with private key K_A^- (m signed is a combination of m itself and a digital signature of m, in this case a digital signature signed with K_A^- implemented by encrypting the cryptographic hash value of m with K_A^-): $$\{m\}^{K_A^-} \rightsquigarrow \left\{m, Sig\{m\}^{K_A^-}\right\} \rightsquigarrow \left\{m, \left\{H\{m\}\right\}_{K_A^-}\right\}$$
$\{m\}^{[A,B]}$	m is signed with a shared secret of A and B (m signed is a combination of m itself and a digital signature of m, in this case a digital signature signed with a shared secret of A and B): $$\{m\}^{[A,B]} \rightsquigarrow \left\{m, Sig\{m\}^{[A,B]}\right\}$$

$\{m\}^{K_{A,B}}$	m is signed with a shared secret of A and B; the shared key $K_{A,B}$ (m signed is a combination of m itself and a digital signature of m, in this case a digital signature signed with the shared key $K_{A,B}$ implemented by encrypting the cryptographic hash value of m with $K_{A,B}$): $$\{m\}^{K_{A,B}} \rightsquigarrow \left\{m, Sig\{m\}^{K_{A,B}}\right\} \rightsquigarrow \left\{m, \{H\{m\}\}_{K_{A,B}}\right\}$$
$Sign([A], m)$	A signs m (m signed is a combination of m itself and a digital signature of m, in this case a digital signature signed by A implemented by $[A]$ encrypting the cryptographic hash value of m): $$Sign([A], m) \rightarrow \{m\}^{[A]} \rightsquigarrow \left\{m, Sig\{m\}^{[A]}\right\}$$
$Sign(K_A^-, m)$	Sign m with private key K_A^- (m signed is a combination of m itself and a digital signature of m, in this case a digital signature signed with K_A^- implemented by encrypting the cryptographic hash value of m with K_A^-): $$Sign(K_A^-, m) \rightarrow \{m\}^{K_A^-} \rightsquigarrow \left\{m, Sig\{m\}^{K_A^-}\right\} \rightsquigarrow \left\{m, \{H\{m\}\}_{K_A^-}\right\}$$
$Sign([A, B], m)$	Sign m with a shared secret of A and B (m signed is a combination of m itself and a digital signature of m, in this case a digital signature signed with a shared secret of A and B implemented by encrypting the cryptographic hash value of m with a key based on a shared secret of $[A]$ and $[B]$): $$Sign([A, B], m) \rightarrow \{m\}^{[A,B]} \rightsquigarrow \left\{m, Sig\{m\}^{[A,B]}\right\}$$
$Sign(K_{A,B}, m)$	Sign m with a shared secret of A and B; the shared key $K_{A,B}$ (m signed is a combination of m itself and a digital signature of m, in this case a digital signature signed with the shared key $K_{A,B}$ implemented by encrypting the cryptographic hash value of m with $K_{A,B}$): $$Sign(K_{A,B}, m) \rightarrow \{m\}^{K_{A,B}} \rightsquigarrow \left\{m, Sig\{m\}^{K_{A,B}}\right\} \rightsquigarrow \left\{m, \{H\{m\}\}_{K_{A,B}}\right\}$$
$PwKey(P, s)$	Create a secret key from the password P with the salt s. The salt is optional in the notation. *PBKDF2* is an examples of a password-based key derivation function. The function creates a new secret key: $$PwKey(P_1) \rightarrow K''_{P_1}$$ $$PwKey_{PBKDF2}(P_2, s) \rightarrow K''_{P_2}$$
$DHPubKey(K_A^-, p)$	Make a public key K_A^+ (a key share) from the private key K_A^- and the public parameters p, typically used in a Diffie–Hellman key exchange protocol [16] (a Diffie–Hellman key share). The public parameters argument is optional in the notation: $$DHPubKey(K_A^-) \rightarrow K_A^+$$ $$DHPubKey(K_B^-, p) \rightarrow K_B^+$$ In the original implementation of Diffie-Hellman (Finite Field Diffie–Hellman) the public parameters consists of a agreed upon prime and a primitive root.

$DHKey(K_A^-, K_B^+, p)$	Combine private key K_A^- with public key K_B^+ and the public parameters p (optional) to generate a new secret (shared) key $K_{A,B}$, typically the result of a Diffie–Hellman key exchange protocol [16]. The public parameters argument is optional: $DHKey(K_A^-, K_B^+) \rightarrow K_{A,B}$ $DHKey(K_B^-, K_A^+, p) \rightarrow K_{A,B}$ These functions are one-way functions where the private keys K_A^- and K_B^- can not be calculated (in reasonable time) with only the knowledge of the public keys K_A^+ and K_B^+ and the public parameters p. As a consequence, the new shared secret key $K_{A,B}$ is also difficult (impossible in practice) to calculate.
$Verify(K_A^+, s)$	Verify that the signed structured value (message) s is signed by the matching private key K_A^- of public key K_A^+ and, as a consequence, verify that s is signed by A: $Verify(K_A^+, s) \rightsquigarrow Verify(K_A^+, \{m\}^{[x]}) \rightsquigarrow Verify(K_A^+, \{m, Sig\{m\}^{[x]}\}) \rightsquigarrow Verify(K_A^+, \{m, Sig\{m\}^{K_x^-}\}) \rightsquigarrow Verify(K_A^+, \{m, \{H\{m\}\}_{K_x^-}\})$: $\left.\begin{array}{r} Decrypt(K_A^+, Sig\{m\}^{[x]}) = H\{m\} \rightsquigarrow \\ Decrypt(K_A^+, Sig\{m\}^{K_x^-}) = H\{m\} \rightsquigarrow \\ Decrypt(K_A^+, \{H\{m\}\}_{K_x^-}) = H\{m\} \end{array}\right\} \Leftrightarrow K_A^- = K_x^-$ $Verify(K_A^+, s) \rightarrow true \Leftrightarrow K_A^- = K_x^-$
$Verify([C], s)$	Verify that the signed structured value (message) s is signed by principal C.
$Cert\{A, K_A^+\}^{[C]}$	A certificate where certificate authority C binds identity A to public key K_A^+ (in the example, $\ldots$ is other certificate related meta-data): $Cert\{A, K_A^+\}^{[C]} \rightsquigarrow \{A, K_A^+, \ldots\}^{[C]}$
$Cert\{A, K_A^+\}^{K_C^-}$	A certificate where a certificate authority with private key K_C^- binds identity A to public key K_A^+ (in the example, $\ldots$ is other certificate related meta-data): $\frac{Cert\{A, K_A^+\}^{K_C^-} \rightsquigarrow \{A, K_A^+, \ldots\}^{K_C^-}}{Verify(K_C^+, Cert\{A, K_A^+\}^{K_C^-}) \rightsquigarrow Verify(K_C^+, \{A, K_A^+, \ldots\}^{K_C^-}) \rightarrow true}$

2.2 BAN logic

The description below of the BAN logic notation is copied directly, with some minor modifications, from the original paper presenting the BAN logic, "A Logic of Authentication" [12].

Notation	Description
$A \mid\equiv X$	A *believes* X, or A would be entitled to believe X. In particular the principal A may act as though X is true. This construct is central to the BAN logic.
$A \triangleleft X$	A *sees* X. Someone has sent a message containing X to A, who can read and repeat X possibly after doing some decryption.

$A \mid\sim X$	A *once said* X. The principal A at some time sent a message including the statement X. It is not known whether the message was sent long ago or during the current run of the protocol, but it is known that A believed X when A sent the message.
$A \Rightarrow X$	A has *jurisdiction* over X (A *controls* X). The principal A is an authority on X and should be trusted on this matter. This construct is used when a principal has delegated authority over some statement. For example, encryption keys need to be generated with some care, and in some protocols certain servers are trusted to do this properly. This may be expressed by the assumption that the principals believe that the server has jurisdiction over statements about the quality of keys.
$\sharp(X)$	The formula X is *fresh*, that is, X has not been sent in a message at any time before the current run of the protocol. This is usually true for nonces, that is expressions generated for the purpose of being fresh (nonce—number used once). Nonces commonly include a timestamp or a number that is used only once, such as a sequence number.
$A \overset{K}{\longleftrightarrow} B$	A and B may use the *shared key* K to communicate. The key K is good, in that it will never be discovered by any principal except A or B, or a principal trusted by either A or B. (In Aspen, a shared key A and B may use to communicate can be denoted $K_{A,B}$.)
$\overset{K}{\longmapsto} A$	A has K as a *public key*. The matching secret key (the inverse of K, denoted K^{-1}) will never be discovered by any principal except A, or a principal trusted by A. (In Aspen, a public key of A can be denoted K_A^+, and the inverse of K_A^+, the private key, is denoted K_A^-.)
$A \overset{X}{\Longleftrightarrow} B$	The formula X is a *secret* known only to A and B, and possibly to principals trusted by them. Only A and B may use X to prove their identities to one another. Often X is fresh as well as secret. An example of a shared secret is a password.
$\{X\}_K$	This represents the formula X *encrypted* under the key K. Formally, $\{X\}_K$ is an abbreviation for an expression of the form $\{X\}_K$ *from* A. We make the realistic assumption that each principal is able to recognize and ignore his own messages; the originator of each message is mentioned for this purpose. In the interests of brevity, we typically omit this in our examples.
$\langle X \rangle_Y$	This represents X *combined* with the formula Y; it is intended that Y be a secret, and that its presence prove the identity of whoever utters $\langle X \rangle_Y$. In implementations, X is simply concatenated with the password Y; our notation highlights that Y plays a special rôle, as proof of origin for X. The notation is intentionally reminiscent of that for encryption, which also guarantees the identity of the source of a message through knowledge of a certain kind of secret.

In the Aspen notation, when we write $K_{A,B}$, it is implicit that A and B may use $K_{A,B}$ to communicate. We can use the BAN logic notation to make it explicit:

$$A \overset{K_{A,B}}{\longleftrightarrow} B$$

In a similar manner, K_A^+ is in the Aspen notation implicit a *public key* of A. We can use the BAN logic notation to make it explicit:

$$\overset{K_A^+}{\longmapsto} A$$

Both the BAN logic notation and Aspen use the notation $\{m\}_K$ for the formula m *encrypted* under the key K (m encrypted with the key K). In this case, Aspen has adopted the notation used in BAN logic and in a lot of other related publications and text books.

3 Use the notation in text

This section explains how to use Aspen in LaTeX documents. The new LaTeX commands and environments used are defined in the LaTeX package `aspen`.

We will in the text include notation examples that might not make sense in a security protocol perspective. However, they are included for completeness. We will in the documentation try to include a wide range of possibilities available from the LaTeX package `aspen`.

For commands with arguments, the argument types are given using a notation inspired by the `xparse` argument specification:

`m`	Mandatory arguments *Examples:* `\cmd{arg}`
`o`	Optional arguments *Examples:* `\cmd`, `\cmd[arg]`
`O{default}`	Optionals with default value *Examples:* `\cmd`, `\cmd[arg]`
`p`	Optionals in parenthesis *Examples:* `\cmd`, `\cmd(arg)`
`P{default}`	Parenthesis optionals with default *Examples:* `\cmd`, `\cmd(arg)`
`t'`	Optional single quote marks *Examples:* `\cmd`, `\cmd'`
`B`	Optional bracket sizes (`big`, `Big`, ...) *Examples:* `\cmd`, `\cmd[Big]`
`s`	Optional stars (alternative versions) *Examples:* `\cmd`, `\cmd*`
`T`	Optional key types: `*`, `-`, `+`, `!`, `'`, or `"` *Examples:* `\cmd`, `\cmd-`, `\cmd!`
`x`	Optionals with magic return *Examples:* `\cmd`, `\cmd[*]`, `\cmd[arg]`
`i`	Optional embellishment (more details) *Examples:* `\cmd`, `\cmd_{arg}`
→	Arguments of returned command *Examples:* `\mkcmd`: `omo` → `Bimx`

We can use this notation to specify the type of the arguments to a command. For example, `om` says that the command takes two arguments where the first one is optional (in square brackets). We use the symbol → to specify the arguments of a command created by another command. For example, `\AMktval` has one optional argument (`o`) and one mandatory argument (`m`) and returns a new command that has one optional star argument (`s`), one optional bracket size argument (`B`), one optional embellishment (`i`), and one mandatory argument (`m`):

```
\AMktval: om → sBim
```

The optional magic return type `x` is available for some predefined functions and can be be made available for functions created with the commands `\AMkfunc` and `\AMkkfunc`. The magic return value `*` will create the return value based on what the function does. For example, if magic return is used with the encrypt function, the return value created will be the encrypted value:

```
\AEncrypt: TBimmx
\AEncrypt+{A}{m}[*]   →   Encrypt(K_A^+, m) → {m}_{K_A^+}
```

3.1 Aspen

The table below lists the Aspen notation with the matching LaTeX commands. More examples of the usage of the notation are found in Section 4.

Notation	LaTeX code and description
$=, <, \leq, >, \geq$	`=`,`<`,`\leq`,`>`,`\geq`, used to compare values.
$\oplus$, .	`\AXor`, `\AConcat`, used as binary operators for *exclusive or* and *concatenation* (used to concatenate two values or strings), respectively.
$\Rightarrow, \Leftrightarrow$	`\AIfthen`,`\AIffthen`, used to reason about protocols and protocol steps (meaning, *"if, then"* and *"if, and only if, then"*, respectively).
1, 2	`\AVal{1}`,`\AVal{2}`, used for values.
Arguments:	`\AVal`: `m` `\AVal{<value>}`
$v \leftarrow 0$	`\AAssign{v}{0}`, used to assign a name to a value. In this example, the value 0 is assigned to the name v.
Arguments:	`\AAssign`: `mm` `\AAssign{<name>}{<value>}`
true,*false*	`\ATrue`,`\AFalse`, used for the boolean values.

e **?** $c : s$ **;;** ...	`\ACond[e]{\ACondstep{c}{s}\ANextbreak\ldots}`, used to change the behavior of a protocol based on a condition. A conditional consists of an optional expression e followed by question mark **?** followed by conditional steps. Each conditional step consist of a condition c followed by a colon : followed by the step s that should be performed if the condition is true. Conditional steps are separated by a single or a double semicolon, **;** (`\ANextcont`, continue if previous condition is true) or **;;** (`\ANextbreak`, break if previous condition is true). In the following example, N'_A is assigned the value of N'_x if N'_x is equal to N'_y, otherwise the protocol is aborted:
	`\ACond[\ANonce{x}=\ANonce{y}]{%`
	`  \ACondstep{\ATrue}{\AAssign{\ANonce{A}}{\ANonce{x}}}%`
	`  \ANextbreak`
	`  \ACondstep{\AFalse}{\AAbort}}`
	→ $N'_x = N'_y$ **?** *true* : $N'_A \leftarrow N'_x$ **;;** *false* : *Abort*()

Arguments:	`\ACond`: `om`, `\ACondstep`: `mm` `\ACond[<expression>]{<cond-steps>}`, `\ACondstep{<cond>}{<step>}`
$\wedge, \vee, \neg$	`\land`, `\lor` and `\lnot`, respectively. Can be used in the conditions of conditional steps.
$x \rightsquigarrow y$	`x \AUnpack y`, used to unpack more details.
A, B, S	`\APri{A}`,`\APri{B}`,`\APri{S}`, used for principals.
Arguments:	`\APri`: `m` `\APri{<principal>}`
N'_A, N'_S	`\ANonce*{N_A}`,`\ANonce{S}`, used for nonces. The `\ANonce` command has an optional first argument to change the symbol (the letter): `\ANonce[n]{0}` → n'_0

Arguments: `\ANonce`: `sO{N}m`
`\ANonce<*>[<symbol>]{<id>}`

C_A, I_B `\ACounter*{C_A}`,`\ACounter{B}`, used for indexes or counters. The `\ACounter` command has an optional first argument to change the symbol (the letter): `\ACounter[i]{0}` → i_0

Arguments: `\ACounter`: `sO{I}m`
`\ACounter<*>[<symbol>]{<id>}`

R_x, R_y, R'_1 `\ARandom*{R_x}`,`\ARandom{y}`,`\ARandom'{1}`, used for random values (the ′ hints about limited useful lifetime). The `\ARandom` command has an optional first argument to change the symbol (the letter): `\ARandom[r]{z}` → r_z

Arguments: `\ARandom`: `st'O{R}m`
`\ARandom<*><'>[<symbol>]{<id>}`

T_A, T_S, T_L, L_1 `\ATS*{T_A}`,`\ATS{S}`,`\ATTL*{L}`,`\ATTL{1}`, used for time related values, like time stamps and lifetime (time to live). Both the `\ATS` and `\ATTL` command have an optional first argument to change the symbol (the letter): `\ATS[t]{0}` → t_0, `\ATTL[l]{1}` → l_1

Arguments: `\ATS`: `sO{T}m`, `\ATTL`: `sO{L}m`
`\ATS<*>[<symbol>]{<id>}`, `\ATTL<*>[<symbol>]{<id>}`

"Hello" `\AStr{Hello}`, used for text strings.

Arguments: `\AStr`: `m`
`\AStr{<str>}`

---

x, y `\AVar{x}`,`\AVar{y}`, used for variables.

Arguments: `\AVar`: `m`
`\AVar{<variable>}`

$^-, ^+, ', \ldots$ Some command markers (key type markers) are used throughout the Aspen LaTeX package to specify the key type involved:

- `*` : Means no specific variant (argument is the key, not the label): `\AKey*{K}`
- `-` : Mark that it is a private key (from a public-private key pair): `\AKey-{A}`
- `+` : Mark that it is a public key (from a public-private key pair): `\AKey+{A}`
- `!` : Mark by principal instead of key (the key of): `\AKey!{A}`
- `'` : Mark that it is temporary (session key or limited lifetime): `\AKey'{A}`
- `"` : Mark that it is a key generated from the password: `\AKey"{P}`

`\AKey*{K}`	→	K
`\AEncrypted"{P}{\AKey-{B}}`	→	$\{K_B^-\}_{K_P''}$
`\ASig-{A}{m}`	→	$Sig\{m\}^{K_A^-}$
`\AEncrypt+{B}{m}`	→	$Encrypt(K_B^+, m)$
`\ASigned!{S}{m}`	→	$\{m\}^{[S]}$
`\ADecrypt'{A,B}{c}`	→	$Decrypt(K'_{A,B}, c)$

Arguments: `T` (the symbol used for these optional markers in argument specifications)

K `\AKey*{K}`, used for (non-specific) encryption keys. The argument is the key itself, and not a label for the key symbol.

Arguments: `\AKey`: `Tm`
`\AKey*{<key>}`

$K_A, K_{A,B}$	`\AKey{A}`,`\ASharedkey{A,B}`, used for shared (secret/symmetric) keys (provided by two different LaTeX commands, where the first is a more compact version; use whatever you prefer). The `\ASharedkey` command has an optional first argument to change the symbol (the letter): `\ASharedkey[k]{B}` → k_B
Arguments:	`\AKey`: `Tm`, `\ASharedkey`: `O{K}m` `\AKey{`*`<id>`*`}`, `\ASharedkey[`*`<symbol>`*`]{`*`<id>`*`}`
$K'_{C,S}, K'_{A,B,S}$	`\AKey'{C,S}`,`\ASessionkey{A,B,S}`, used for session keys (provided by two different LaTeX commands, where the first is a more compact version; use whatever you prefer). The `\ASessionkey` command has an optional first argument to change the symbol (the letter): `\ASessionkey[k]{A,B}` → $k'_{A,B}$
Arguments:	`\AKey`: `Tm`, `\ASessionkey`: `O{K}m` `\AKey'{`*`<id>`*`}`, `\ASessionkey[`*`<symbol>`*`]{`*`<id>`*`}`
K^+_A, K^+_B	`\AKey+{A}`,`\APubkey{B}`, used for public keys (provided by two different LaTeX commands, where the first is a more compact version; use whatever you prefer). The `\APubkey` command has an optional first argument to change the symbol (the letter): `\APubkey[k]{S}` → k^+_S
Arguments:	`\AKey`: `Tm`, `\APubkey`: `O{K}m` `\AKey+{`*`<id>`*`}`, `\APubkey[`*`<symbol>`*`]{`*`<id>`*`}`
K^-_A, K^-_B	`\AKey-{A}`,`\APrivkey{B}`, used for private keys (provided by two different LaTeX commands, where the first is a more compact version; use whatever you prefer). The `\APrivkey` command has an optional first argument to change the symbol (the letter): `\APrivkey[k]{A}` → k^-_A
Arguments:	`\AKey`: `Tm`, `\APrivkey`: `O{K}m` `\AKey-{`*`<id>`*`}`, `\APrivkey[`*`<symbol>`*`]{`*`<id>`*`}`
$K^{''}_{P_1}, K^{''}_{P_2}$	`\AKey"{P_1}`,`\APwkey{P_2}`, used for keys generated from a password (provided by two different LaTeX commands, where the first is a more compact version; use whatever you prefer). The `\APwkey` command has an optional first argument to change the symbol (the letter): `\APwkey[k]{P}` → $k^{''}_P$
Arguments:	`\AKey`: `Tm`, `\APwkey`: `O{K}m` `\AKey"{`*`<password>`*`}`, `\APwkey[`*`<symbol>`*`]{`*`<password>`*`}`
$[A]$	`\AName{A}`, typically used to indicate who signed (or encrypted) a message, but no specific key is given, known or relevant. If you mark a key with a `!`, the produced output is the same: `\AKey!{A}` → $[A]$
Arguments:	`\AName`: `m`, `\AKey`: `Tm` `\AName{`*`<id>`*`}`, `\AKey!{`*`<id>`*`}`
M_1–M_n	`\AGroup{M}`, specifies a group and is typically used as a label for a shared key shared within a group with n members: `\AKey{\AGroup{M}}` → $K_{M_1-M_n}$ `\AGroup[O][s]{M}`, used when th group member indexes are non-standard: `\AKey{\AGroup[O][s]{M}}` → $K_{M_0-M_s}$ `\AGroup*{M}`, typically used in a text when referring to a group (with n members): `\AKey{\AGroup*{M}}` → $K_{M_1,\ldots,M_n}$

`\AGroup*[0][s]{M}`, used when group member indexes are non-standard:

`\AKey{\AGroup*[0][s]{M}}` → $K_{M_{0,\ldots,M_s}}$

Arguments: `\AGroup`: `sO{1}O{n}m`
`\AGroup<*>[<first>][<last>]{<id>}`

$\{m\}$, $\{A, B\}$ `\ASval{m}`,`\AMsg{\APri{A},\APri{B}}`, used for a structured value or message (a message can be seen as structured value).

Arguments: `\ASval`: `Bm`, `\AMsg`: `Bm`
`\ASval[<size>]{<value>}`, `\AMsg[<size>]{<message>}`

$\{m\}$ `\ASval[big]{m}`, or `\AMsg[big]{m}`, where first optional size argument can be `big`, `Big`, `bigg`, or `Bigg` for increased size of parenthesis (typically used with nested structured values and/or functions):

`\ASval{x}` → $\{x\}$

`\ASval[big]{\ASval{x}}` → $\{\{x\}\}$

`\ASval[Big]{\ASval[big]{\ASval{x}}}` → $\{\{\{x\}\}\}$

`\ASval[bigg]{\ASval[Big]{\ASval[big]{\ASval{x}}}}` → $\{\{\{\{x\}\}\}\}$

We can even use bigger size specifiers: `big`, `Big`, `bigg`, `Bigg`, `biggg`, `Biggg`, `bigggg`, `Bigggg`, `biggggg`, and `Biggggg`:

$$\{\{\{\{\{\{\{\{\{\{x\}\}\}\}\}\}\}\}\}\}$$

The optional size argument applies for all Aspen LaTeX commands that produces a pair of parenthesis, both ordinary parenthesis and curly brackets. A few examples (see below for more details on these commands):

`\ATval[big]{Type}{m}` → $Type\{m\}$

`\ASend[big]{A}{B}{m}` → $A \longrightarrow B : \{m\}$

`\AFunc[big]{Func}{x,y}` → $Func(x, y)$

`\AEncrypted[big]{A,B}{m}` → $\{m\}_{K_{A,B}}$

Arguments: `\ASval`: `Bm`, `\AMsg`: `Bm`
`\ASval[<size>]{<value>}`, `\AMsg[<size>]{<message>}`

… `_{…}`, where the embellishment is used to provide more details or information about a structured value or a function. A few examples where the embellishments are *MD5*, *RSA*, *AES*, *DSA*, and *SHA-2*:

`\AChash_{MD5}{m}` → $H_{MD5}\{m\}$

`\AEncrypt+_{RSA}{B}{m}` → $Encrypt_{RSA}(K_B^+, m)$

`\ADecrypt'_{AES}{A,B}{c}` → $Decrypt_{AES}(K'_{A,B}, c)$

`\ASig-_{DSA}{S}{m}` → $Sig_{DSA}\{m\}^{K_S^-}$

`\AHmac_{SHA-2}{A}{m}` → $HMAC_{SHA\text{-}2}\{m\}^{K_A}$

Arguments: `i` (the symbol used for such optional embellishment in argument specifications)

Type{*m*}	`\ATval{Type}{m}`, used for a typed structured value where the first argument is the type. The `\ATval*` variant is used for a typed structured value where the first argument is the type, but the value is not wrapped with curly brackets. This is typically used when the value is already wrapped as a structured value (e.g., encrypted or signed data). This is an example with a Kerberos Authenticator as a typed structured value: `\ATval*{KA}{\AEncrypted{S,C}{\APri{C},\textit{Addr}_C,\ATS{t}}}` → $KA\{C, Addr_C, T_t\}_{K_{S,C}}$ The embellishment (`_{RSA}` in the example below) can be used to give more details about the typed structured value: `\ATval*{KA}_{RSA}{\AEncrypted{S,C}{\APri{C},\textit{Addr}_C,\ATS{t}}}` → $KA_{RSA}\{C, Addr_C, T_t\}_{K_{S,C}}$
Arguments:	`\ATval`: `sBmim` `\ATval<*>[<size>]{<type>}_{<embellishment>}{<value>}`
$A \longrightarrow B : \{m\}$	`\ASend{A}{B}{m}`, used to specify that a message {*m*} is sent from *A* to *B*. The `\ASend*` variant is used to specify that the message is not wrapped as a structured value or message. This is typically used when what-is-sent is already wrapped as a structured value (e.g., encrypted or signed data): `\ASend*{A}{B}{\AEncrypted+{B}{m}}` → $A \longrightarrow B : \{m\}_{K_B^+}$ `\ASend*{A}{B}{\AEncrypted+[big]{B}{\AChash{m}}}` → $A \longrightarrow B : \{H\{m\}\}_{K_B^+}$
Arguments:	`\ASend`: `sBmmm` `\ASend<*>[<size>]{<sender>}{<receiver>}{<message>}`
Func(*x*, *y*)	`\AFunc{Func}{x,y}`, used for any functions. An optional last argument is used if a return value of the function is given: `\AFunc{Func}{x,y}[z]` → $Func(x, y) \rightarrow z$
Arguments:	`\AFunc`: `Bmimo` `\AFunc[<size>]{<name>}_{<embellishment>}{<arguments>}[<returns>]`
Proc(“text”)	`\AProc{Proc}[text]`, used for any procedures. The optional last argument is a comment (describing why this procedure is called).
Arguments:	`\AProc`: `P{white}mio` `\AProc(<bg-color>){<name>}_{<embellishment>}[<comment>]`
Abort(“text”)	`\AAbort[text]`, used for the abort procedure. The optional argument is a comment (describing why the protocol is aborted).
Arguments:	`\AProc`: `P{white}io` `\AAbort(<bg-color>)_{<embellishment>}[<comment>]`

$\{m\}_{K_{A,B}}$	`\AEncrypted{A,B}{m}`, where the message is encrypted with an shared secret encryption key (in this case, the shared key $K_{A,B}$ of A and B). The other options (with markers) are: `\AEncrypted*{K}{m}` → $\{m\}_{K}$ `\AEncrypted+{A}{m}` → $\{m\}_{K_A^+}$ `\AEncrypted-{A}{m}` → $\{m\}_{K_A^-}$ `\AEncrypted!{A}{m}` → $\{m\}_{[A]}$ `\AEncrypted'{A,B}{m}` → $\{m\}_{K'_{A,B}}$ `\AEncrypted"{P}{m}` → $\{m\}_{K_P''}$
Arguments:	`\AEncrypted`: `TBimm` `\AEncrypted<T>[<size>]_{<embellishment>}{<key>}{<plain>}`
$Encrypt(K_{A,B}, m)$	`\AEncrypt{A,B}{m}`, where the value m is encrypted with a secret shared encryption key (in this case, a shared key of A and B). Other options are `*`, `+`, `-`, `!`, `'` and `"` (see above for explanation). Since `\AEncrypt` is a function, we can include a return value as an optional last argument (and it supports magic return making straightforward to include the return value): `\AEncrypt+{B}{m}[\AEncrypted+{B}{m}]` → $Encrypt(K_B^+, m) \to \{m\}_{K_B^+}$ `\AEncrypt+{B}{m}[*]` → $Encrypt(K_B^+, m) \to \{m\}_{K_B^+}$
Arguments:	`\AEncrypt`: `TBimmx` `\AEncrypt<T>[<size>]_{<embellishment>}{<key>}{<plain>}[<returns>]`
$Decrypt(K_{A,B}, c)$	`\ADecrypt{A,B}{\AVar{c}}`, where the cipher text c is decrypted with an secret shared encryption key (in this case, a shared key between A and B). Other options are `*`, `+`, `-`, `! '`, and `"` (see above for explanation). Since `\ADecrypt` is a function, we can include a return value as an optional last argument: `\ADecrypt-{B}{\AEncrypted+{B}{m}}[m]` → $Decrypt(K_B^-, \{m\}_{K_B^+}) \to m$
Arguments:	`\ADecrypt`: `TBimmo` `\ADecrypt<T>[<size>]_{<embellishment>}{<key>}{<cipher>}[<returns>]`
$H\{m\}$	`\AChash{m}`, used for a cryptographic hash value of m.
Arguments:	`\AChash`: `Bim` `\AChash[<size>]_{<embellishment>}{<value>}`
$MAC\{m\}^{K_A}$	`\AMac{A}{m}`, used for message authentication code of m with K_A.
Arguments:	`\AMac`: `TBimm` `\AMac<T>[<size>]_{<embellishment>}{<key>}{<value>}`
$CMAC\{m\}^{K_A}$	`\ACmac{A}{m}`, used for cipher-based message authentication code of m with K_A.
Arguments:	`\ACmac`: `TBimm` `\ACmac<T>[<size>]_{<embellishment>}{<key>}{<value>}`
$HMAC\{m\}^{K_A}$	`\AHmac{A}{m}`, used for HMAC message authentication code of m with K_A.
Arguments:	`\AHmac`: `TBimm` `\AHmac<T>[<size>]_{<embellishment>}{<key>}{<value>}`

$H(m)$	`\AChashf{m}`, used for a cryptographic hash value function producing the cryptographic hash value of m. Since `\AChashf` is a function, we can include a return value as an optional last argument: `\AChashf{m}[*]` → $H(m) \rightarrow H\{m\}$
Arguments:	`\AChashf`: `Bimo` `\AChasf[<size>]_{<embellishment>}{<value>}[<returns>]`
$MAC(K_A, m)$	`\AMacf{A}{m}`, used for a message authentication code function with the arguments K_A and m. Since `\AMacf` is a function, we can include a return value as an optional last argument: `\AMacf{A}{m}[*]` → $MAC(K_A, m) \rightarrow MAC\{m\}^{K_A}$
Arguments:	`\AMacf`: `TBimmo` `\AMacf<T>[<size>]_{<embellishment>}{<key>}{<value>}[<returns>]`
$CMAC(K_A, m)$	`\ACmacf{A}{m}`, used for a cipher-based message authentication code with the arguments K_A and m. Since `\ACmacf` is a function, we can include a return value as an optional last argument: `\ACmacf{A}{m}[*]` → $CMAC(K_A, m) \rightarrow CMAC\{m\}^{K_A}$
Arguments:	`\ACmacf`: `TBimmo` `\ACmacf<T>[<size>]_{<embellishment>}{<key>}{<value>}[<returns>]`
$HMAC(K_A, m)$	`\AHmacf{A}{m}`, used for a HMAC message authentication code with the arguments K_A and m. Since `\AHmacf` is a function, we can include a return value as an optional last argument: `\AHmacf{A}{m}[*]` → $HMAC(K_A, m) \rightarrow HMAC\{m\}^{K_A}$
Arguments:	`\AHmacf`: `TBimmo` `\AHmacf<T>[<size>]_{<embellishment>}{<key>}{<value>}[<returns>]`
$Sig\{m\}^{K_A^-}$	`\ASig-{A}{m}`, used for the signature of A on m, where the `-` says that the signature is signed with a private key (in this case, the private key of A). The other key type markers can also be used with this command.
Arguments:	`\ASig`: `TBimm` `\ASig<T>[<size>]_{<embellishment>}{<key>}{<value>}`
$Sig(K_A^-, m)$	`\ASigf-{A}{m}`, used to create a signature of A on m, where the `-` says that the signature is signed with a private key (in this case, the private key of A). The other key type markers can also be used with this command. Since `\ASigf` is a function, we can include a return value as an optional last argument (and it supports magic return making straightforward to include the return value).
Arguments:	`\ASigf`: `TBimmx` `\ASigf<T>[<size>]_{<embellishment>}{<key>}{<value>}[<returns>]`
$\{m\}^{K_A^-}$	`\ASigned-{A}{m}`, used for m signed, where the `-` says that the signature is signed with a private key (in this case, the private key of A). The other key type markers can also be used with this command.
Arguments:	`\ASigned`: `TBimm` `\ASigned<T>[<size>]_{<embellishment>}{<key>}{<value>}`

$Sign(K_A^-, m)$	`\ASign-{A}{m}`, used to sign m, where the `-` says that the signature is signed with a private key (in this case, the private key of A). The other key type markers can also be used with this command. Since `\ASign` is a function, we can include a return value as an optional last argument (and it supports magic return making straightforward to include the return value).
Arguments:	`\ASign`: `TBimmx` `\ASign<T>[<size>]_{<embellishment>}{<key>}{<value>}[<returns>]`
$PwKey(P, s)$	`\APwkeyf{P}(s)`, used to create a secret key from a password P. Optionally, a salt value s can be provided Since `\APwkeyf` is a function, we can include a return value as an optional last argument (and it supports magic return making straightforward to include the return value): `\APwkeyf{P_1}(s)[*]` → $PwKey(P_1, s) \to K''_{P_1}$ `\APwkeyf{P_2}[x]` → $PwKey(P_2) \to x$
Arguments:	`\APwkeyf`: `sBimpx` `\APwkeyf<*>[<size>]_{<embellishment>}{<password>}(<salt>)[<returns>]`
$DHPubKey(K_A^-, p)$	`\ADhpubkeyf{A}(p)`, used to create a public key K_A^+ (a key share) from a private key K_A^- and the optional public parameters p, typically used in a Diffie–Hellman key exchange protocol [16]. `\ADhpubkeyf{A}(p)[A]` → $DHPubKey(K_A^-, p) \to K_A^+$ `\ADhpubkeyf*{x}[y]` → $DHPubKey(x) \to y$
Arguments:	`\ADhpubkeyf`: `sBimpo` `\ADhpubkeyf<*>[<size>]_{<embellishment>}{<key>}(<parms>)[<returns>]`
$DHKey(K_A^-, K_B^+, p)$	`\ADhkeyf{A}{B}(p)`, used to combine a private key K_A^- with public key K_B^+ and the optional public parameters p to generate a new secret (shared) key $K_{A,B}$, typically used in a Diffie–Hellman key exchange protocol [16]. `\ADhkeyf{A}{B}(p)[A,B]` → $DHKey(K_A^-, K_B^+, p) \to K_{A,B}$ `\ADhkeyf*{x}{y}[z]` → $DHKey(x, y) \to z$
Arguments:	`\ADhkeyf`: `sBimmpo` `\ADhkeyf<*>[<size>]_{<embellishment>}{<key>}{<key>}(<parms>)[<returns>]`
$Verify(K_A^+, s)$	`\AVerify+{A}{\AVar{s}}`, used to verify the signed data s, where the `+` says that the signed data is verified towards the public key of A. The other key type markers can also be used with this command.
Arguments:	`\AVerify`: `TBimmo` `\AVerify<T>[<size>]_{<embellishment>}{<key>}{<value>}[<returns>]`
$Cert\{B, K_B^+\}^{[C]}$	`\ACertificate!{C}{\APri{B},\AKey+{B}}`, used for a certificate binding the public key K_B^+ (public key of B) to the principal B, where `!` says that the signature is signed by the certificate authority C. The other key type markers can also be used with this command.
Arguments:	`\ACertificate`: `TBimm` `\ACertificate<T>[<size>]_{<embellishment>}{<key>}{<content>}`

$Cert\{A, K_A^+\}^{K_C^-}$	`\ACert-{C}{A}`, used for a certificate binding the public key K_A^+ (public key of A) to the principal A, where the `-` says that the signature is signed with a private key (in this case, the private key of the certificate authority C). The other key type markers can also be used with this command.
Arguments:	`\ACert`: `TBimm` `\ACert<T>[<size>]_{<embellishment>}{<key>}{<principal>}`

$X\{m\}$	`\AMktval{X}`, used to create a new typed structured value type where the argument is the type. In this example, the result is a new LaTeX command `\ATvalX` (created combining the prefix `ATval` and the given name). We can for example use this to create a new typed structured type for a specific message type:
	`\AMktval{ReqMsg}`
	`\ATvalReqMsg{\APri{A},m}` → $ReqMsg\{A, m\}$
	The new command will have a `*` version similar to the `\ATval*` command (the value is not wrapped with curly brackets). The `\AMktval` has an optional first argument to specify the name of the command created:
	`\AMktval[reqmsg]{RMsg}`
	`\reqmsg{m}` → $RMsg\{m\}$

Arguments:	`\AMktval`: `om` → `sBim`
	`\AMktval[<cmd>]{<type>}`
	→ `\cmd<*>[<size>]_{<embellishment>}{<value>}`

$X\{m\}_{K_A}$	`\AMketval{X}`, used to create a new *encrypted* typed structured value type where the argument is the type. In this example, the result is a new LaTeX command `\AEtvalX` (created combining the prefix `AEtval` and the given name). We can for example use this to create a new typed structured type for an encrypted message type:
	`\AMketval{EMsg}`
	`\AEtvalEMsg'{C,S}{m}` → $EMsg\{m\}_{K'_{C,S}}$
	The `\AMketval` has an optional first argument to specify the name of the command created (here we define the command `\aka` for Kerberos Authenticators):
	`\AMketval[aka]{KA}`
	`\aka'{S,C}{\APri{C},\textit{Addr}_C,\ATS{s}}` → $KA\{C, Addr_C, T_s\}_{K'_{S,C}}$
	In this case, it might be a good idea to create a new LaTeX command `\ka` implemented with `\aka` and the proper arguments (implementation details not shown):
	`\newcommand{\ka}[3]{\aka'{...}}`
	`\ka{S,C}{C}{s}` → $KA\{C, Addr_C, T_s\}_{K'_{S,C}}$

Arguments:	`\AMketval`: `om` → `TBimm`
	`\AMketval[<cmd>]{<type>}`
	→ `\cmd<T>[<size>]_{<embellishment>}{<key>}{<value>}`

$X\{m\}^{K_A}$

`\AMkstval{X}`, used to create a new *signed* typed structured value type where the argument is the type. In this example, the result is a new LaTeX command `\AStvalX` (created combining the prefix `AStval` and the given name). We can for example use this to create a new typed structured type for an signed message type:

```
\AMkstval{SMsg}
\AStvalSMsg-{A}{m}
```
→ $SMsg\{m\}^{K_A^-}$

The `\AMkstval` has an optional first argument to specify the name of the command created:

```
\AMkstval[smsg]{SMsg}
\smsg-{S}{m}
```
→ $SMsg\{m\}^{K_S^-}$

Arguments:

`\AMkstval`: `om` → `TBimm`
`\AMkstval[<cmd>]{<type>}`
→ `\cmd<T>[<size>]_{<embellishment>}{<key>}{<value>}`

$X(x,y)$

`\AMkfunc{X}`, used when creating a new function type where the argument is the name of the function type. In this example the result is a new LaTeX command `\AFuncX` (created combining the prefix `AFunc` and the given name). We can for example use this to create a new function type for a creating a Kerberos Authenticator:

```
\AMkfunc{KA}
\AFuncKA{\APri{C},\textit{Addr}_C,\ATS{s}}
```
→ $KA(C, Addr_C, T_s)$

The `\AMkfunc` has an optional first argument to specify the name of the command created:

```
\AMkfunc[kaf]{KA}
\kaf{\APri{C},\textit{Addr}_C,\ATS{s}}
```
→ $KA(C, Addr_C, T_s)$

The `\AMkfunc` has an optional third and last argument to specify the name of the command used for magic return:

```
\AMktval[aset]{Set}]
\AMkfunc[asetf]{Set}[aset]
\asetf{\AVal{x},\AVal{y}}[*]
```
→ $Set(x,y) \rightarrow Set\{x,y\}$

Arguments:

`\AMkfunc`: `omo` → `Bimx`
`\AMkfunc[<cmd>]{<name>}[<magic-return>]`
→ `\cmd[<size>]_{<embellishment>}{<arguments>}[<returns>]`

$X(K_A,x,y)$

`\mkkfunk{X}`, used when creating a new function type for functions where the first argument is an encryption key. The argument is the name of the function type. In this example the result is a new LaTeX command `\AKfuncX` (created combining the prefix `AKfunc` and the given name) with two arguments; the first argument is an encryption key and the second argument is a comma separated list of the rest of the function arguments. We can for example use this to create this new function type with an encryption key as the first argument (a session key in this example):

```
\AMkkfunc{KeyF}
\AKfuncKeyF'{A}{\textit{Addr},\ATS{s}}
```
→ $KeyF(K'_A, Addr, T_s)$

The `\AMkkfunc` has an optional first argument to specify the name of the command created:

```
\AMkkfunc[kf]{KeyF}
\kf'{A}{\textit{Addr},\ATS{s}}
```
→ $KeyF(K'_A, Addr, T_s)$

The `\AMkkfunc` has an optional third and last argument to specify the name of the command used for magic return:

```
\AMkstval[sset]{SSet}]
\AMkkfunc[ssetf]{SignSet}[sset]
\ssetf-{A}{\AVal{x},\AVal{y}}[*]
```
→ $SignSet(K_A^-, x, y) \to SSet\{x, y\}^{K_A^-}$

Arguments:
`\AMkkfunc`: `omo` → `TBimmx`
`\AMkkfunc[`*`<cmd>`*`]{`*`<name>`*`}[`*`<magic-return>`*`]`
→ `\`*`cmd<T>`*`[`*`<size>`*`]_{`*`<embellishment>`*`}{`*`<key>`*`}{`*`<arguments>`*`}[`*`<returns>`*`]`

X()

`\AMkproc{X}`, used when creating a new procedure type where the argument is the name of the procedure type. In this example the result is a new LaTeX command `\AProcX` (created combining the prefix `AProc` and the given name). We can for example use this to create a new procedure type for protocol failures:

```
\AMkproc{Fail}
\AProcFail[message]
```
→ *Fail*("message")

The `\AMkproc` has an optional argument to specify the name of the command created:

```
\AMkproc[fail]{Fail}
\fail[message]
```
→ *Fail*("message")

All procedures has an optional first argument in parentheses (before the optional message) that specifies the background color. Its default value is white, but you should set this to the correct color if the background where the name of the procedure is printed is not white. The reason is that the name is printed with a contour color to make the underline look nicer, especially when characters break the line:

```
\AMkproc[syncfailure]{SyncFailure}
\syncfailure[without correct background color set]\newline
\syncfailure(LightLightGrey)[with correct background color set]
```
→ *SyncFailure*("without correct background color set")
SyncFailure("with correct background color set")

Arguments:
`\AMkproc`: `om` → `P{white}io`
`\AMkproc[`*`<cmd>`*`]{`*`<name>`*`}`
→ `\`*`cmd(<bg-color>)_{<embellishment>}[<message>]`*

3.2 BAN logic

The table below lists the BAN logic notation with the matching LaTeX commands. This notation is available when the LaTeX package `aspen` is loaded with the option `ban`.

Notation	LaTeX code and description
$\mid\!\equiv$	`\BANBelieves`, used to state that someone *believes* something (and acts as it is true): `\APri{A}\BANBelieves\AVal{X}` → $A \mid\!\equiv X$
$\triangleleft$	`\BANSees`, used to state that someone sees something (Someone has sent a message to someone and they have bee able to read it): `\APri{A}\BANSees\AVal{X}` → $A \triangleleft X$
$\mid\!\sim$	`\BANOncesaid`, used to state to someone at some time said something (someone some time sent a message including the statement): `\APri{A}\BANOncesaid\AVal{X}` → $A \mid\!\sim X$
$\models\!\!\Rightarrow$	`\BANControls`, used to state that someone has *jurisdiction* (controls) over something: `\APri{A}\BANControls\AVal{X}` → $A \models\!\!\Rightarrow X$
$\sharp(X)$	`\BANFresh{X}`, used to state that something is fresh (X has not been sent in a message at any time before in the current run of the protocol).
$\overset{K}{\longleftrightarrow}$	`\BANSharedkey{K}`, used to state that a key is shared: `\APri{A}\BANSharedkey{\AKey{A,B}}\APri{B}` → $A \overset{K_{A,B}}{\longleftrightarrow} B$
$\overset{K}{\longmapsto}$	`\BANPubkey{K}`, used to state that a key is a public key of someone: `\BANPubkey{\AKey+{A}}\APri{A}` → $\overset{K_A^+}{\longmapsto} A$
$\overset{X}{\leftrightharpoons}$	`\BANSecret{X}`, used to state that a secret (X, in this case) is only known to them: `\APri{A}\BANSecret{X}\APri{B}` → $A \overset{X}{\leftrightharpoons} B$
$\{X\}_K$	`\BANEncryptedwith{K}{X}`, used to state that something is encrypted with the key (X is encrypted with the key K).
$\langle x \rangle_y$	`\BANCombine{x}{y}`, used to state that x is combined with y: `\BANCombine{\AVal{X}}{\AVal{Y}}` → $\langle X \rangle_Y$

3.3 Series of steps

The LaTeX package `aspen` provides support for presenting a security protocol as a series of messages and steps with the `ASteps` environment. A message between to principals is in the `ASteps` environment typeset with the familiar `\ASend` command. With `\ASend` commands, the `ASteps` environment can be used like this:

```
\begin{ASteps}
  \ASend*{A}{B}{\AEncrypted+{B}{m_1}}[m1] \\
  \ASend*{B}{A}{\AEncrypted+{A}{m_2}}[m2]
\end{ASteps}
```

$M_1 \quad A \longrightarrow B : \{m_1\}_{K_B^+}$

$M_2 \quad B \longrightarrow A : \{m_2\}_{K_A^+}$

Notice that each step is separated by the `\\` command. Each step is labeled and can be referred to by its name (`\ARef{m1}` $\rightarrow M_1$, and `\ARef{m2}` $\rightarrow M_2$). The `ASteps*` version of the environment is without the labels:

```
\begin{ASteps*}
  \ASend*{A}{B}{\AEncrypted+{B}{m_1}} \\
  \ASend*{B}{A}{\AEncrypted+{A}{m_2}}
\end{ASteps*}
```

$A \longrightarrow B : \{m_1\}_{K_B^+}$

$B \longrightarrow A : \{m_2\}_{K_A^+}$

By default, the `ASteps` environment has two types of labels; `M` for messages and `S` for other steps. In the example above only messages (`\ASend` commands) are used. Other steps are given with the `\AStep` or the `\AStepat` commands. In the following example the `\AStep` command is used and the space between the label and the step is adjusted with the optional key-value argument `lspace` (the default value is `1.5em`):

```
\begin{ASteps}[lspace=1em]
  \AStep{\AEncrypt+{B}{m_1}[*]} \\
  \AStep{\ASign-[big]{A}{%
      \AEncrypted+{B}{m_1}}[*]}
\end{ASteps}
```

$S_1 \quad Encrypt(K_B^+, m_1) \rightarrow \{m_1\}_{K_B^+}$

$S_2 \quad Sign\left(K_A^-, \{m_1\}_{K_B^+}\right) \rightarrow \left\{\{m_1\}_{K_B^+}\right\}^{K_A^-}$

The optional key-value argument `rmarg` sets the right margin width of the ASteps environment and `lmarg` sets the left margin width of the ASteps environment. The default margin widths are `\tabcolsep`. In the following example the margins are removed:

```
\begin{ASteps*}[lmarg=0pt,rmarg=0pt]
  \AStep{No margins}
\end{ASteps*}
```

No margins

We can also change the margins, and the space between the label and the step, by adjusting the lengths `\AStepsleftmargin`, `\AStepsrightmargin` and `\AStepslabelspace`. To change these values for the whole document we can place these commands at the beginning of the LaTeX file (after the LaTeX package `aspen` is loaded):

```
\setlength{\AStepsleftmargin}{0pt}
\setlength{\AStepsrightmargin}{0pt}
\setlength{\AStepslabelspace}{1em}
```

The `\AStepat` command can be used to specified *where* a step is performed. The command has an extra first argument where this is specified (in this example, at principal A):

```
\begin{ASteps}*
  \AStepat{A}{\ASign-{A}{m_1}[*]} \\
  \AStepat{A}{\AEncrypt+[big]{B}{%
      \ASigned-{A}{m_1}}[*]}
\end{ASteps}
```

$S_3 \quad A : Sign(K_A^-, m_1) \rightarrow \{m_1\}^{K_A^-}$

$S_4 \quad A : Encrypt\left(K_B^+, \{m_1\}^{K_A^-}\right) \rightarrow \left\{\{m_1\}^{K_A^-}\right\}_{K_B^+}$

The * marker of the ASteps environment (not to be confused with the ASteps* version of the environment) means that the counters of the labels are *not* reset (the counting continues from the previous ASteps environment). It is also possible to set the value of each counter using the standard LaTeX command \setcounter. For example, if you want the counter of the S labels to start with zero, you add this as the first command inside a ASteps environment: \setcounter{counterS}{-1} (see B.3.13 for an example).

It is also possible to add new types of labels with the optional key-value argument labels. In this example, new label types A and B are introduced and the \AStepat commands are labeled with the new label types by using the optional first argument to the command:

```
\begin{ASteps}[labels={A,B}]
  \AStepat(A){A}{\AEncrypt+{B}{m_1}[*]} \\
  \ASend*{A}{B}{\AEncrypted+{B}{m_1}} \\
  \AStepat(B){B}{\ADecrypt-[big]{B}{%
      \AEncrypted+{B}{m_1}}[m_1]}
\end{ASteps}
```

A_1 $A : Encrypt(K_B^+, m_1) \rightarrow \{m_1\}_{K_B^+}$

M_1 $A \longrightarrow B : \{m_1\}_{K_B^+}$

B_1 $B : Decrypt(K_B^-, \{m_1\}_{K_B^+}) \rightarrow m_1$

The \AStepat* version of the \AStepat command changes the horizontal position of the text of such steps so the colons are aligned:

```
\begin{ASteps*}
  \ASend*{A}{B}{\ASigned-{A}{m_1}} \\
  \AStepat*{B}{\AVerify+{A}{\ASigned-{A}{m_1}}}
\end{ASteps*}
```

$A \longrightarrow B : \{m_1\}^{K_A^-}$

$B : Verify(K_A^+, \{m_1\}^{K_A^-})$

The \AStep* version of the \AStep command changes the horizontal position of the text of the step in a similar way:

```
\begin{ASteps*}
  \ASend*{A}{B}{\ASigned-{A}{m_1}} \\
  \AStep*{\AVerify+{A}{\ASigned-{A}{m_1}}}
\end{ASteps*}
```

$A \longrightarrow B : \{m_1\}^{K_A^-}$

$Verify(K_A^+, \{m_1\}^{K_A^-})$

The \ARawstep command is a low-level command that usually is not necessary. In a ASteps* environment it has four optional arguments followed by one non-optional argument. In a ASteps environment another optional first argument and an optional last argument is added related to the labels of the step. To better understand the command we show it here used together with a \ASend command in a ASteps environment:

```
\begin{ASteps}
  \ASend{A}{B}{m}[s1] \\
  \ARawstep(M)[a][--][b][;]{m}[s2]
\end{ASteps}
```

M_1 $A \longrightarrow B : \{m\}$

M_2 a – b ; m

The arguments of the commands in the ASteps* environment

\ASend: sBmmm
\ASend<*>[<*size*>]{<*sender*>}{<*receiver*>}{<*message*>}

\AStep: sm
\AStep<*>{<*step*>}

\AStepat: smm
\AStepat<*>{<*where*>}{<*step*>}

\ARawstep: oooom
\ARawstep[*<sender>*][*<arrow>*][*<receiver>*][*<colon>*]{*<step>*}

The arguments of the commands in the ASteps environment

\ASend: sP{M}Bmmmo
\ASend<*>(*<type>*)[*<size>*]{*<sender>*}{*<receiver>*}{*<message>*}[*<label>*]

\AStep: sP{S}mo
\AStep<*>(*<type>*){*<step>*}[*<label>*]

\AStepat: sP{S}mmo
\AStepat<*>(*<type>*){*<where>*}{*<step>*}[*<label>*]

\Rawstep: P{S}oooomo
\ARawstep(*<type>*)[*<sender>*][*<arrow>*][*<receiver>*][*<colon>*]{*<step>*}[*<label>*]

4 Notation usage examples

To illustrate the usability of the notation, we provide a few examples where the notation is used to describe well-known, and not so well-known, security protocols. In the original papers referred to below, you will find the original notations used. The inconsistencies in the notations used in these papers are a major motivation behind Aspen. The following examples are presented here:

- *Original Needham–Schroeder protocol* — LaTeX code: B.3.1
- *Revised Needham–Schroeder protocol* — LaTeX code: B.3.2
- *Otway-Rees protocol* — LaTeX code: B.3.3
- *Kerberos protocol* — LaTeX code: B.3.4
- *Diffie–Hellman key exchange* — LaTeX code: B.3.5
- *Needham–Schroeder public key protocol* — LaTeX code: B.3.6
- *Needham–Schroeder-Lowe public key protocol* — LaTeX code: B.3.7
- *ASW protocol* — LaTeX code: B.3.8
- *Wide-mouthed-frog protocol* — LaTeX code: B.3.9
- *Idealized wide-mouthed-frog protocol* — LaTeX code: B.3.10
- *Wide-mouthed-frog protocol with declarations* — LaTeX code: B.3.11
- *TLS 1.3 Handshake* — LaTeX code: B.3.12
- *SMC: Calculate the mean value* — LaTeX code: B.3.13
- *File sharing with symmetric and public key encryption* — LaTeX code: B.3.14
- *Scalable secure file sharing* — LaTeX code: B.3.15

The three last examples listed are from my own publications. I have included them since they represent the start of my process developing this notation. You will find that my use of notations then was a mixed bag without consistency across publications.

Original Needham–Schroeder protocol

The *Needham–Schroeder protocol* [18] aims to establish a session key between two parties on a network, typically to protect further communication. The protocol is based on a symmetric encryption and it forms the basis for the Kerberos protocol (the *Needham–Schroeder public key protocol* is presented on page 28).

Original Needham–Schroeder protocol

$$
\begin{aligned}
M_1 \quad & A \longrightarrow S : \{A, B, N'_A\} \\
M_2 \quad & S \longrightarrow A : \left\{N'_A, B, K_{A,B}, \{K_{A,B}, A\}_{K_{B,S}}\right\}_{K_{A,S}} \\
M_3 \quad & A \longrightarrow B : \{K_{A,B}, A\}_{K_{B,S}} \\
M_4 \quad & B \longrightarrow A : \{N'_B\}_{K_{A,B}} \\
M_5 \quad & A \longrightarrow B : \{N'_B - 1\}_{K_{A,B}}
\end{aligned}
$$

N'_A and N'_B are nonces and the shared key $K_{A,B}$ should be fresh, $\sharp(K_{A,B})$. The protocol is vulnerable to a replay attack [15]. See B.3.1 for the LaTeX code.

Revised Needham–Schroeder protocol

The *Revised Needham–Schroeder protocol* [19] addresses a weakness in the protocol related to its vulnerable to a replay attack. A fun fact regarding this suggested revision is its link to my home department, Department of Computer Science at UiT[2]. From [19]:

[2] UiT The Arctic University of Norway, formerly University of Tromsø

> *In 1986 one of us (RMN) gave a lecture at the University of Tromsø which included the 1978 protocol, the criticism of it, and also a general principle about the use of nonce identifiers. This was that the identifier should always be generated by the party that sought reassurance about the time integrity of a transaction. In discussion Dr Sape J. Mullender of CWI Amsterdam pointed out that this should apply to the reassurance of B against the attack outlined.*

The revised version adds two initial messages, M_1 and M_2, between the two principals A and B and the extra nonce N_I' as an identifier of the session:

Revised Needham–Schroeder protocol

$$
\begin{aligned}
M_1 \quad & A \longrightarrow B : \{A\} \\
M_2 \quad & B \longrightarrow A : \{A, N_I'\}_{K_{B,S}} \\
M_3 \quad & A \longrightarrow S : \left\{A, B, N_A', \{A, N_I'\}_{K_{B,S}}\right\} \\
M_4 \quad & S \longrightarrow A : \left\{N_A', B, K_{A,B}, \{K_{A,B}, A, N_I'\}_{K_{B,S}}\right\}_{K_{A,S}} \\
M_5 \quad & A \longrightarrow B : \{K_{A,B}, A, N_I'\}_{K_{B,S}} \\
M_6 \quad & B \longrightarrow A : \{N_B'\}_{K_{A,B}} \\
M_7 \quad & A \longrightarrow B : \{N_B' - 1\}_{K_{A,B}}
\end{aligned}
$$

The inclusion of the new nonce N_I' prevents any replaying of compromised versions of message $\{K_{A,B}, A\}_{K_{B,S}}$ since the revised version of the message contains N_I': $\{K_{A,B}, A, N_I'\}_{K_{B,S}}$. This can not be forged since an attacker does not have $K_{B,S}$. See B.3.2 for the LaTeX code of the protocol.

Otway-Rees protocol

The *Otway-Rees protocol* [20] is essential the same as the Revised Needham-Schroeder protocol:

Otway-Rees protocol

$$
\begin{aligned}
M_1 \quad & A \longrightarrow B : \left\{I_A, A, B, \{N_A', I_A, A, B\}_{K_{A,S}}\right\} \\
M_2 \quad & B \longrightarrow S : \left\{I_A, A, B, \{N_A', I_A, A, B\}_{K_{A,S}}, \{N_B', I_A, A, B\}_{K_{B,S}}\right\} \\
M_3 \quad & S \longrightarrow B : \left\{I_A, \{N_A', K_{A,B}'\}_{K_{A,S}}, \{N_B', K_{A,B}'\}_{K_{B,S}}\right\} \\
M_4 \quad & B \longrightarrow A : \left\{I_A, \{N_A', K_{A,B}'\}_{K_{A,S}}\right\}
\end{aligned}
$$

The identifier I_A prevents the replay attack since an attacker is not able to alter $\{N_A', I_A, A, B\}_{K_{A,S}}$ and $\{N_B', I_A, A, B\}_{K_{B,S}}$. See B.3.3 for the LaTeX code.

Kerberos protocol

The *Kerberos protocol* [23] is based on the Needham-Schroeder protocol, but makes use of timestamps as nonces to remove the problems of the original Needham-Schroeder protocol and to reduce the number of messages needed:

Kerberos protocol

$$
\begin{aligned}
M_1 \quad & A \longrightarrow S : \{A, B\} \\
M_2 \quad & S \longrightarrow A : \left\{T_s, L, K_{A,B}, B, \{T_s, L, K_{A,B}, A\}_{K_{B,S}}\right\}_{K_{A,S}} \\
M_3 \quad & A \longrightarrow B : \left\{\{T_s, L, K_{A,B}, A\}_{K_{B,S}}, \{A, T_a\}_{K_{A,B}}\right\} \\
M_4 \quad & B \longrightarrow A : \{T_a + 1\}_{K_{A,B}}
\end{aligned}
$$

T_s and T_a are timestamps and L is a lifetime. See B.3.4 for the LaTeX code.

Diffie–Hellman key exchange

The *Diffie–Hellman key exchange* protocol [16] is used to establish a shared symmetric key between two participants over a public channel based a secret (a private key) at each participant and some shared public parameters:

Diffie–Hellman key exchange

$$
\begin{array}{lll}
M_1 & A \longrightarrow B : \{p\} \\
S_1 & A : K_A^+ \leftarrow DHPubKey(K_A^-, p) \\
S_2 & B : K_B^+ \leftarrow DHPubKey(K_B^-, p) \\
M_2 & A \longrightarrow B : \{K_A^+\} \\
M_3 & B \longrightarrow A : \{K_B^+\} \\
S_3 & A : K_{A,B} \leftarrow DHKey(K_A^-, K_B^+, p) \\
S_4 & B : K_{A,B} \leftarrow DHKey(K_B^-, K_A^+, p)
\end{array}
$$

The public keys created in step S_1 and S_2 are also called Diffie-Hellman key shares. At the end of this protocol the two participants A and B has a shared secret key $K_{A,B}$. In BAN logic, we can state the following about the outcome $K_{A,B}$:

$$A \overset{K_{A,B}}{\longleftrightarrow} B$$

In step M_1 above, participant A send the public parameters p to participant B. Other means of agreeing on these parameters are possible and not important for the protocol itself. See B.3.5 for the LaTeX code of the *Diffie–Hellman key exchange* protocol.

Needham–Schroeder public key protocol

The *Needham–Schroeder public key protocol* [18] intends to provide mutual authentication between two parties:

Needham–Schroeder public key protocol

$$
\begin{array}{lll}
M_1 & A \longrightarrow S : \{A, B\} \\
M_2 & S \longrightarrow A : \{K_B^+, B\}^{K_S^-} \\
M_3 & A \longrightarrow B : \{N_A', A\}_{K_B^+} \\
M_4 & B \longrightarrow S : \{B, A\} \\
M_5 & S \longrightarrow B : \{K_A^+, A\}^{K_S^-} \\
M_6 & B \longrightarrow A : \{N_A', N_B'\}_{K_A^+} \\
M_7 & A \longrightarrow B : \{N_B'\}_{K_B^+}
\end{array}
$$

This protocol is vulnerable to a man-in-the-middle attack [17]. The fix is however easy.

Needham–Schroeder-Lowe public key protocol

To avoid a man-in-the-middle attack, the *Needham–Schroeder-Lowe public key protocol* [17] includes the identity of the responder in message M_6 of the protocol:

Needham–Schroeder-Lowe public key protocol

$$
\begin{array}{ll}
M_1 & A \longrightarrow S : \{A, B\} \\
M_2 & S \longrightarrow A : \{K_B^+, B\}^{K_S^-} \\
M_3 & A \longrightarrow B : \{N'_A, A\}_{K_B^+} \\
M_4 & B \longrightarrow S : \{B, A\} \\
M_5 & S \longrightarrow B : \{K_A^+, A\}^{K_S^-} \\
M_6 & B \longrightarrow A : \{N'_A, N'_B, B\}_{K_A^+} \\
M_7 & A \longrightarrow B : \{N'_B\}_{K_B^+}
\end{array}
$$

See B.3.6 and B.3.7 for the LaTeX code of the two versions of the *Needham–Schroeder public key protocol*.

ASW protocol

The *ASW protocol* is an optimistic fair-exchange protocol for contract signing [8]. This is a good example for Aspen since we in publications find very different (and, if I may say so, hard to read) notations used when presenting the protocol [8, 11]. This is a simplified version of the *exchange* subprotocol (the main part of the protocol) of ASW is shown in Aspen:

ASW exchange protocol

$$
\begin{array}{ll}
M_1 & O \longrightarrow R : \left\{K_A^+, K_B^+, m, H\{N'_O\}\right\}^{K_O^-} \\
M_2 & R \longrightarrow O : \left\{\left\{K_A^+, K_B^+, m, H\{N'_O\}\right\}^{K_O^-}, H\{N'_R\}\right\}^{K_R^-} \\
M_3 & O \longrightarrow R : \{N'_O\} \\
M_4 & R \longrightarrow O : \{N'_R\}
\end{array}
$$

Two participants O (originator) and R (recipient) is involved in this subprotocol. In the complete protocol two other subprotocols (*abort* and *resolve*) and a third participant T (third player) is included. See B.3.8 for the LaTeX code.

Wide-mouthed-frog protocol

The *Wide-mouthed-frog protocol* [12] (Section 7, page 25) is a simple protocol that uses shared key cryptography and an authentication server. It transfers a key from A to B via the authentication server S in only two messages by using synchronized clocks and by allowing A to choose the session key:

Wide-mouthed-frog protocol

$$
\begin{array}{ll}
M_1 & A \longrightarrow S : \left\{A, \{T_A, B, K'_{A,B}\}_{K_{A,S}}\right\} \\
M_2 & S \longrightarrow B : \{T_S, A, K'_{A,B}\}_{K_{B,S}}
\end{array}
$$

A sends a time stamp T_A and session key $K'_{A,B}$ to S. S checks that message M_1 is timely. If it is, it forwards the key $K'_{A,B}$ to B together with its own timestamp T_S in message M_2. B then checks that the timestamp T_S from S is later than any another it has received from S.

Idealized wide-mouthed-frog protocol

The *Idealized wide-mouthed-frog protocol* with BAN logic is shown below:

Idealized wide-mouthed-frog protocol

$$
\begin{array}{ll}
M_1 & A \longrightarrow S : \left\{T_A, \{A \overset{K_{A,B}}{\longleftrightarrow} B\}\right\}_{K_{A,S}} \\
M_2 & S \longrightarrow B : \{T_S, A \mid\equiv A \overset{K_{A,B}}{\longleftrightarrow} B\}_{K_{B,S}}
\end{array}
$$

Wide-mouthed-frog protocol with declarations

In [11], formal declarations as part of protocol narrations is introduced. We can do something similar with BAN logic and Aspen, where we use BAN logic for the declaration part (D_1–D_2). The following example is similar to their version of the *Wide-mouthed-frog protocol with declarations* (see [11], Table 3, page 487):

Wide-mouthed-frog protocol with declarations

$$
\begin{array}{ll}
D_1 & A \overset{K_{A,S}}{\longleftrightarrow} S;\ B \overset{K_{B,S}}{\longleftrightarrow} S \\
D_2 & A \triangleleft K'_{A,B};\ \sharp(K'_{A,B});\ A \triangleleft m \\
M_1 & A \longrightarrow S : \left\{A, \{T_A, B, K'_{A,B}\}_{K_{A,S}}\right\} \\
M_2 & S \longrightarrow B : \{T_S, A, K'_{A,B}\}_{K_{B,S}}
\end{array}
$$

See B.3.9, B.3.10 and B.3.11 for the LaTeX code of the *Wide-mouthed-frog protocol*, the *Idealized wide-mouthed-frog protocol* and the *Wide-mouthed-frog protocol with declarations*.

TLS 1.3 Handshake

The *TLS 1.3 Handshake* [21] has three stages. Stage one is the key exchange, stage two is the server parameters, and stage three is authentication (we only include server authentication in the example). In the steps below stage one and three are included, the key exchange algorithm used is DHE (Ephemeral Diffie-Hellman) and we ignore PSK (Pre-Shared Key):

TLS 1.3 Handshake

$$
\begin{array}{lll}
S_1 & & C : K^+_{C_e} \leftarrow DHPubKey(K^-_{C_e}),\ \sharp(K^-_{C_e}) \\
M_1 & C \longrightarrow S : & \{\ldots, N'_C, K^+_{C_e}, \ldots\} \\
S_2 & & S : K^+_{S_e} \leftarrow DHPubKey(K^-_{S_e}),\ \sharp(K^-_{S_e}) \\
M_2 & S \longrightarrow C : & \left\{\ldots, N'_S, K^+_{S_e}, Cert\{S, K^+_S\}^{K^-_{CA}}, \{N'_{C'}, N'_{S'}, \ldots\}^{K^-_S}, \ldots\right\} \\
S_3 & & C : Verify\left(K^+_{CA}, Cert\{S, K^+_S\}^{K^-_{CA}}\right)\ ?\ false : \underline{Abort}() \\
S_4 & & C : Verify(K^+_S, \{N'_C, N'_S, \ldots\}^{K^-_S})\ ?\ false : \underline{Abort}() \\
S_5 & & C : N'_C = N'_{C'} \wedge N'_S = N'_{S'}\ ?\ false : \underline{Abort}() \\
S_6 & & C : K_{C,S} \leftarrow DHKey(K^-_{C_e}, K^+_{S_e}) \\
M_3 & C \longrightarrow S : & \{\ldots\}_{K_{C,S}} \\
S_7 & & S : K_{C,S} \leftarrow DHKey(K^-_{S_e}, K^+_{C_e})
\end{array}
$$

The example above is somewhat simplified to make it easier to follow (some details are hidden and some optional pathways are ignored). Message M_1 is the *client hello* message and message M_2 is both the *server hello* and the *server finished* message. After steps S_6 and S_7 the client C and the server S share a new fresh encryption key $K_{C,S}$ and the rest of the TLS handshake is encrypted. The *client finished* message M_3 concludes the handshake and the client and the server can securely communicate with the shared key $K_{C,S}$.

SMC: Calculate the mean value

In [4], *SMC* (secure multi-party computation) algorithms for analyzing health data were discussed. The first algorithm from this paper is used to calculate the mean value of three values V_1, V_2 and V_3 from three participants P_1, P_2 and P_3 without sharing any knowledge about the individual values. A coordinator C coordinates the calculation. The messages between from P_{i-1} to P_i have the following structure (we can say that participant P_0 and P_4 is the same individual with an alias C):

$$\left\{\left\{\{V_i'\}^{K^-_{P_{i-1}}}\right\}_{K^+_{P_i}}, \left\{\{P_{i-1}, P_{i+1}\}^{K^-_C}\right\}_{K^+_{P_i}}, \left\{\{P_i, P_{i+2}\}^{K^-_C}\right\}_{K^+_{P_{i+1}}}, \ldots\right\}^{K^-_C}$$

The first part of the message is the intermediate value received at participant P_i signed by the previous participant in the calculation P_{i-1}. The second part is each path in the calculation (where the input came from and where to send the intermediate result). The current participant P_i can decrypt the first element that says the it should expect the input value from participant P_{i-1} and it should forwards the intermediate result of its calculation to participant P_{i+1}. Each such element is signed by the coordinator and encrypted with the public key of the participant that should be able to read this information. We can now write the protocol used for the calculation of the mean value M using the Aspen notation (we ignore the signature verification steps at each participant):

SMC: Calculate the mean value

$$
\begin{array}{lll}
S_0 & & C : V_0' \leftarrow R_0' \\
M_1 & C \longrightarrow & P_1 : \left\{\left\{\{V_0\}^{K^-_C}\right\}_{K^+_{P_1}}, \left\{\{C, P_2\}^{K^-_C}\right\}_{K^+_{P_1}}, \left\{\{P_1, P_3\}^{K^-_C}\right\}_{K^+_{P_2}}, \left\{\{P_2, C\}^{K^-_C}\right\}_{K^+_{P_3}}, \left\{\{C, _\}^{K^-_C}\right\}_{K^+_C}\right\}^{K^-_C} \\
S_1 & & P_1 : \mathit{Verify}\left(K^+_C, \{C, P_2\}^{K^-_C}\right) \wedge \mathit{Verify}\left(K^+_C, \{\ldots\}^{K^-_C}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \\
S_2 & & P_1 : \mathit{Verify}\left(K^+_C, \{V_0'\}^{K^-_C}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \ ;;\ \mathit{true} : V_1' \leftarrow V_0' + V_1 \\
M_2 & P_1 \longrightarrow & P_2 : \left\{\left\{\{V_1'\}^{K^-_{P_1}}\right\}_{K^+_{P_2}}, \left\{\{P_1, P_3\}^{K^-_C}\right\}_{K^+_{P_2}}, \left\{\{P_2, C\}^{K^-_C}\right\}_{K^+_{P_3}}, \left\{\{C, _\}^{K^-_C}\right\}_{K^+_C}\right\}^{K^-_{P_1}} \\
S_3 & & P_2 : \mathit{Verify}\left(K^+_C, \{P_1, P_3\}^{K^-_C}\right) \wedge \mathit{Verify}\left(K^+_{P_1}, \{\ldots\}^{K^-_{P_1}}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \\
S_4 & & P_2 : \mathit{Verify}\left(K^+_{P_1}, \{V_1'\}^{K^-_{P_1}}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \ ;;\ \mathit{true} : V_2' \leftarrow V_1' + V_2 \\
M_3 & P_2 \longrightarrow & P_3 : \left\{\left\{\{V_2'\}^{K^-_{P_2}}\right\}_{K^+_{P_3}}, \left\{\{P_2, C\}^{K^-_C}\right\}_{K^+_{P_3}}, \left\{\{C, _\}^{K^-_C}\right\}_{K^+_C}\right\}^{K^-_{P_2}} \\
S_5 & & P_3 : \mathit{Verify}\left(K^+_C, \{P_2, C\}^{K^-_C}\right) \wedge \mathit{Verify}\left(K^+_{P_2}, \{\ldots\}^{K^-_{P_2}}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \\
S_6 & & P_3 : \mathit{Verify}\left(K^+_{P_2}, \{V_2'\}^{K^-_{P_2}}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \ ;;\ \mathit{true} : V_3' \leftarrow V_2' + V_3 \\
M_4 & P_3 \longrightarrow & C : \left\{\left\{\{V_3'\}^{K^-_{P_3}}\right\}_{K^+_C}, \left\{\{C, _\}^{K^-_C}\right\}_{K^+_C}\right\}^{K^-_{P_3}} \\
S_7 & & C : \mathit{Verify}\left(K^+_C, \{C, _\}^{K^-_C}\right) \wedge \mathit{Verify}\left(K^+_{P_3}, \{\ldots\}^{K^-_{P_3}}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \\
S_8 & & C : \mathit{Verify}\left(K^+_{P_3}, \{V_3'\}^{K^-_{P_3}}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \ ;;\ \mathit{true} : M \leftarrow (V_3' - V_0')/3
\end{array}
$$

Figure 3 illustrates the participants and each step and message of the calculation. See B.3.13 for the LaTeX code.

File sharing with symmetric and public key encryption

Cloud Security Infrastructure (CSI) [3] is a scalable cloud storage architecture with focus on privacy. In [3], a figure (Figure 3) illustrates how a user can share a file with another user using a combination of symmetric (secret) key and public key encryption. In Figure 4 and in the protocol shown below, the intent of original figure in [3]is recreated using Aspen. In the steps below, A uploads two files F_1 and F_2 to a server, where file F_1 is shared with B:

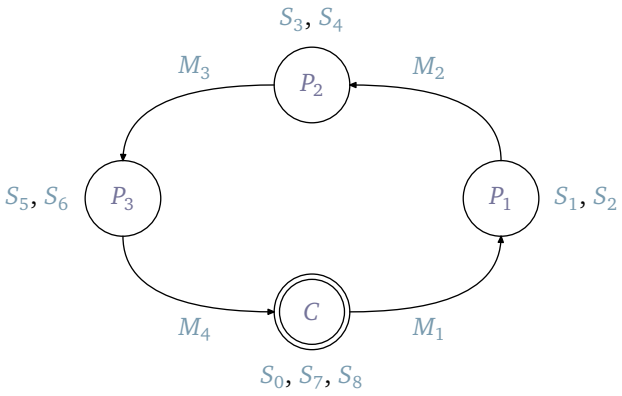

Figure 3: SMC: Calculate the mean value

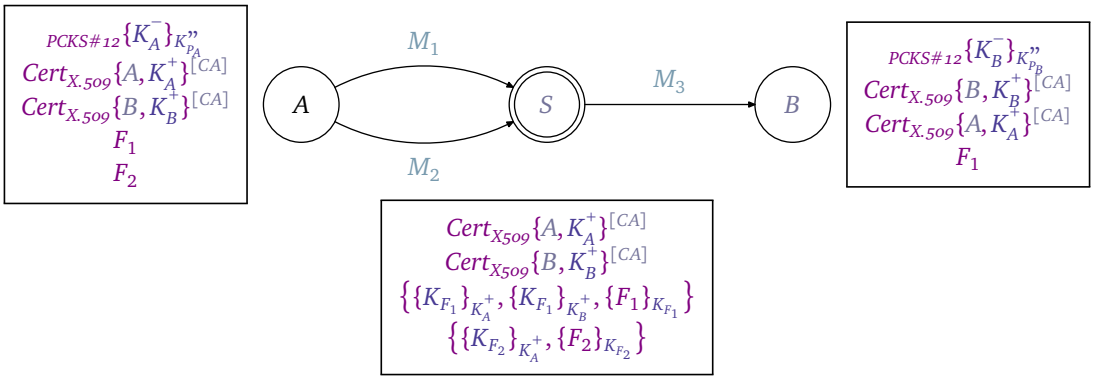

Figure 4: File sharing with symmetric and public key encryption

File sharing with symmetric and public key encryption

S_1		$A : \sharp(K_{F_1}), \sharp(K_{F_2})$
S_2		$A : Encrypt(K_{F_1}, F_1) \rightarrow \{F_1\}_{K_{F_1}}, Encrypt(K_{F_2}, F_2) \rightarrow \{F_2\}_{K_{F_2}}$
S_3		$A : Encrypt(K_A^+, K_{F_1}) \rightarrow \{K_{F_1}\}_{K_A^+}, Encrypt(K_B^+, K_{F_1}) \rightarrow \{K_{F_1}\}_{K_B^+}$
S_4		$A : Encrypt(K_A^+, K_{F_2}) \rightarrow \{K_{F_2}\}_{K_A^+}$
M_1	$A \longrightarrow S$	$: \{\{K_{F_1}\}_{K_A^+}, \{K_{F_1}\}_{K_B^+}, \{F_1\}_{K_{F_1}}\}$
M_2	$A \longrightarrow S$	$: \{\{K_{F_2}\}_{K_A^+}, \{F_2\}_{K_{F_2}}\}$
M_3	$S \longrightarrow B$	$: \{\{K_{F_1}\}_{K_A^+}, \{K_{F_1}\}_{K_B^+}, \{F_1\}_{K_{F_1}}\}$
S_5		$B : Decrypt(K''_{P_B}, \{K_B^-\}_{K''_{P_B}}) \rightarrow K_B^-, Decrypt(K_B^-, \{K_{F_1}\}_{K_B^+}) \rightarrow K_{F_1}$
S_6		$B : Decrypt(K_{F_1}, \{F_1\}_{K_{F_1}}) \rightarrow F_1$

In Figure 4, the boxes illustrate what is stored at each principal (data at-rest) and the arrows illustrate the messages sent between the principals (data in-transit). Each messages in the figure is labeled with the same labels used in the steps shown above. In the ASPEN notation of the protocol above we have included all the encrypt steps S_2–S_4 and decrypt steps S_5–S_6. We usually leave them out since they are explicit given by the content of the messages and the expected usage of the data involved. In this case, we made a point by explicitly including these steps.

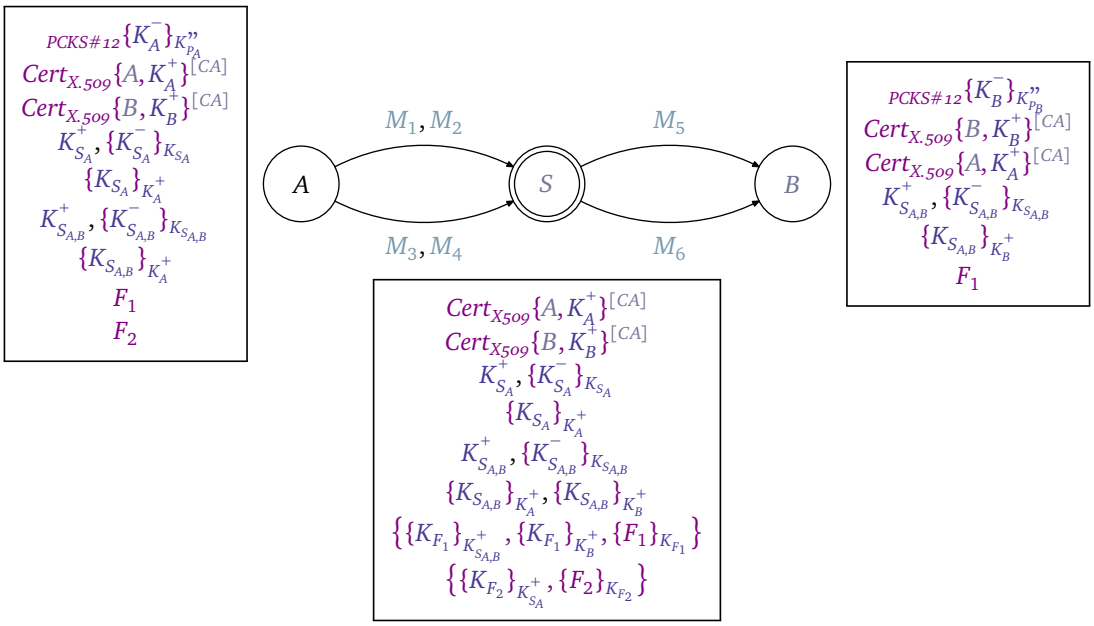

Figure 5: Scalable secure file sharing

Scalable secure file sharing

The Cloud Security Infrastructure (CSI) [3] has come up with the concept of a *store* and a *share* to achieve scalable secure file sharing. Every principal has a store that is represented by a public and private key pair ($K^+_{S_A}$ and $K^-_{S_A}$ for A in the example below). When a file is shared, a new store called a share is created. In the example below a share represented by the public and private key pair $K^+_{S_{A,B}}$ and $K^-_{S_{A,B}}$ is used to share file F_1 between A and B. Each store (and share) also has a symmetric random encryption key associated with them (K_{S_A} and $K_{S_{A,B}}$ in the example) used to encrypt the private keys of the stores:

Scalable secure file sharing

S_1 $\quad A : \sharp(K_{F_1}); \sharp(K_{F_2}); \sharp(K_{S_A}); \sharp(K_{S_{A,B}}); \sharp(K^-_{S_A}); \sharp(K^+_{S_A}); \sharp(K^-_{S_{A,B}}); \sharp(K^+_{S_{A,B}})$

S_2 $\quad A : Encrypt(K_{F_1}, F_1) \rightarrow \{F_1\}_{K_{F_1}}; Encrypt(K_{F_2}, F_2) \rightarrow \{F_2\}_{K_{F_2}}$

S_3 $\quad A : Encrypt(K^+_{S_{A,B}}, K_{F_1}) \rightarrow \{K_{F_1}\}_{K^+_{S_{A,B}}}; Encrypt(K^+_{S_A}, K_{F_2}) \rightarrow \{K_{F_2}\}_{K^+_{S_A}}$

S_4 $\quad A : Encrypt(K_{S_A}, K^-_{S_A}) \rightarrow \{K^-_{S_A}\}_{K_{S_A}}; Encrypt(K_{S_{A,B}}, K^-_{S_{A,B}}) \rightarrow \{K^-_{S_{A,B}}\}_{K_{S_{A,B}}}$

S_5 $\quad A : Encrypt(K^+_A, K_{S_A}) \rightarrow \{K_{S_A}\}_{K^+_A}$

S_6 $\quad A : Encrypt(K^+_A, K_{S_{A,B}}) \rightarrow \{K_{S_{A,B}}\}_{K^+_A}; Encrypt(K^+_B, K_{S_{A,B}}) \rightarrow \{K_{S_{A,B}}\}_{K^+_B}$

M_1 $\quad A \longrightarrow S : \{\{K_{S_A}\}_{K^+_A}, \{K_{S_{A,B}}\}_{K^+_A}, \{K_{S_{A,B}}\}_{K^+_B}\}$

M_2 $\quad A \longrightarrow S : \{\{K^-_{S_A}\}_{K_{S_A}}, \{K^-_{S_{A,B}}\}_{K_{S_{A,B}}}\}$

M_3 $\quad A \longrightarrow S : \{\{K_{F_1}\}_{K^+_{S_{A,B}}}, \{F_1\}_{K_{F_1}}\}$

M_4 $\quad A \longrightarrow S : \{\{K_{F_2}\}_{K^+_{S_A}}, \{F_2\}_{K_{F_2}}\}$

M_5 $\quad S \longrightarrow B : \{\{K_{S_{A,B}}\}_{K^+_B}, \{K^-_{S_{A,B}}\}_{K_{S_{A,B}}}\}$

M_6 $\quad S \longrightarrow B : \{\{K_{F_1}\}_{K^+_{S_{A,B}}}, \{F_1\}_{K_{F_1}}\}$

S_7 $\quad B : Decrypt(K^{"}_{P_B}, \{K^-_B\}_{K^{"}_{P_B}}) \rightarrow K^-_B; Decrypt(K^-_B, \{K_{S_{A,B}}\}_{K^+_B}) \rightarrow K_{S_{A,B}}$

S_8 $\quad B : Decrypt(K_{S_{A,B}}, \{K^-_{S_{A,B}}\}_{K_{S_{A,B}}}) \rightarrow K^-_{S_{A,B}}; Decrypt(K^-_{S_{A,B}}, \{K_{F_1}\}_{K^+_{S_{A,B}}}) \rightarrow K_{F_1}$

S_9 $\quad B : Decrypt(K_{F_1}, \{F_1\}_{K_{F_1}}) \rightarrow F_1$

In Figure 5 presenting the example, the boxes illustrate what is stored at each principal (data at-rest) and the arrows illustrate the messages sent between the principals (data in-transit). Each messages in the figure is labeled with the same labels used in the steps shown above. Figure 5 is a recreation of a figure in [3] (Figure 4) illustrating the same example as the one above.

A References

[1] Abadi, M., Gordon, A.D.: A calculus for cryptographic protocols: The Spi calculus. Information and Computation **148**, 1–70 (1999). 10.1006/inco.1998.2740

[2] Andersen, A.: A LaTeX package for the Aspen security protocol notation and BAN logic. TUGboat **47**(1), 75–80 (2026). 10.47397/tb/47-1/tb145andersen-aspen

[3] Andersen, A., Hardersen, T., Schirmer, N.: Privacy for cloud storage. In: Reimer, H., Pohlmann, N., Schneider, W. (eds.) ISSE 2014 Securing Electronic Business Processes; Highlights of the Information Security Solutions Europe 2014 Conference. Springer-Verlag, Brussels, Belgium (Oct 2014)

[4] Andersen, A., Yigzaw, K.Y., Karlsen, R.: Privacy preserving health data processing. In: Healthcom'14, 16th International Conference on E-health Networking, Application & Services. IEEE, Natal, Brazil (Oct 2014)

[5] Anderson, R.: Security Engineering: A Guide to Building Dependable Distributed Systems. Wiley Computer Publishing, John Wiley & Sons (2001)

[6] Anderson, R.: Security Engineering: A Guide to Building Dependable Distributed Systems. John Wiley & Sons, 2 edn. (2008)

[7] Anderson, R.: Security Engineering: A Guide to Building Dependable Distributed Systems. John Wiley & Sons, 3 edn. (2020)

[8] Asokan, N., Shoup, V., Waidner, M.: Asynchronous protocols for optimistic fair exchange. In: IEEE Symposium on Security and Privacy. pp. 86–99 (1998). 10.1109/SECPRI.1998.674826

[9] Bellare, M., Canetti, R., Krawczyk, H.: Keying hash functions for message authentication. In: Koblitz, N. (ed.) Advances in Cryptology — CRYPTO'96. Lecture Notes in Computer Science, vol. 1109, pp. 1–15. Springer-Verlag (Aug 1996). 10.1007/3-540-68697-5_1

[10] Briais, S., Nestmann, U.: A formal semantics for protocol narrations. In: De Nicola, R., Sangiorgi, D. (eds.) Trustworthy Global Computing, International Symposium, TGC 2005. Lecture Notes in Computer Science, vol. 3705, pp. 163–181. Springer-Verlag, Edinburgh, UK (Apr 2005). 10.1007/11580850_10

[11] Briais, S., Nestmann, U.: A formal semantics for protocol narrations. Theoretical Computer Science **389**(3), 484–511 (Dec 2007). 10.1016/j.tcs.2007.09.005

[12] Burrows, M., Abadi, M., Needham, R.: A logic of authentication. ACM Transactions on Computer Systems **8**(1), 18–36 (Feb 1990). 10.1145/77648.77649

[13] Chappell, D.: Exploring Kerberos, the protocol for distributed security in Windows 2000. Microsoft System Journal (Aug 1999)

[14] Davis, D., Swick, R.: Workstation services and Kerberos authentication at project Athena. LCS Technical Memos MIT-LCS-TM-424, Massachusetts Institute of Technology, Laboratory for Computer Science (Mar 1989)

[15] Denning, D.E., Sacco, G.M.: Timestamps in key distribution protocols. Communications of the ACM **24**(8), 533–536 (Aug 1981). doi.org/10.1145/358722.358740

[16] Diffie, W., Hellman, M.E.: New directions in cryptography. IEEE Transactions on Information Theory **22**(6), 644–654 (Nov 1976)

[17] Lowe, G.: An attack on the Needham-Schroeder public-key authentication protocol. Information Processing Letters **56**(3), 131–136 (Nov 1995). 10.1016/0020-0190(95)00144-2

[18] Needham, R., Schroeder, M.: Using encryption for authentication in large networks of computers. Communications of the ACM **21**(12), 993–999 (Dec 1978). 10.1145/359657.359659

[19] Needham, R., Schroeder, M.: Authentication revisited. Operating Systems Review **21**(1), 7 (Jan 1987). 10.1145/24592.24593

[20] Otway, D., Rees, O.: Efficient and timely mutual authentication. Operating Systems Review **21**(1), 8–10 (Jan 1987). 10.1145/24592.24594

[21] Rescorla, E.: The transport layer security (tls) protocol version 1.3. Request for Comments 8446, Internet Engineering Task Force (IETF), Mozilla (Aug 2018)

[22] Schäfer, G., Festag, A., Karl, H., Wolisz, A.: Current approaches to authentication in wireless and mobile communications networks. TKN Technical Report TKN-01-002, Technical University Berlin, Telecommunication Networks Group (Mar 2001)

[23] Steiner, J.G., Neuman, C., Schiller, J.: Kerberos: An authentication service for open networks systems. In: Proceedings of Usenix Winter Conference 1988. pp. 191–202 (Feb 1988)

B Notes

B.1 Notes on the notation

The notations used for security protocols in different articles and textbooks is not consistent. Aspen is an attempt to create one consistent notation. Mostly, for my own usage, but if the notation is found useful for others, it is a nice bonus. The notation is also influenced by my background as a programmer and not a mathematician or a theoretical computer scientist. In the following, the choices of Aspen will be discussed and compared with similar notations used in articles and textbooks. This is not an attempt to provide a complete overview over existing notations and how they compare to Aspen. It is more a discussion of notations that inspired Aspen and the choices made in the notation. Feedback on the notation are welcome.

Below, Aspen is compared with the notation used in litteratur. The following sources of different notations are used:

1. Aspen
2. Kerberos: An Authentication Service for Open Network Systems [23]
3. A formal semantics for protocol narrations [11]
4. Security Engineering: A Guide to Building Dependable Distributed Systems [7]
5. Current Approaches to Authentication in Wireless and Mobile Communications Networks [22]

Description	**1**	**2**	**3**	**4**	**5**	*
Secret key	K_A	K_A	k_A	K	K_A	C
Shared key	$K_{A,B}$	–	k_{AB}	–	–	C
Session key	$K'_{A,B}$	$K_{A,B}$	–	–	–	B
Public key	K_A^+	–	$\text{pub}(k_A)$	KR	$+K_A$	A
Private key	K_A^-	–	$\text{priv}(k_A)$	KR^{-1}	$-K_A$	A
Encrypted	$\{m\}_K$	$\{m\}K$	$\{m\}_K$	$\{m\}_K$	$\{m\}_K$	C
Signed with	$\{m\}^K$	–	–	$\text{sig}_K\{m\}$	–	A
Signed by	$\{m\}^{[A]}$	–	–	–	$A[m]$	A
Send	$A \rightarrow B : \{m\}$		$A \rightsquigarrow B : m$	$A \rightarrow B : m$	$A \rightarrow B : m$	B
Hash value	$H\{m\}$	–	$\text{H}(m)$	$h(m)$	$H(m)$	B
MAC	$MAC\{m\}^K$	–	–	$\text{MAC}_K(m)$	–	B
HMAC	$HMAC\{m\}^K$	–	–	$\text{HMAC}_K(m)$	–	B
Signature	$Sig\{m\}^K$	–	–	–	–	A
Certificate	$Cert\{A, K_A^+\}^{K_{CA}^-}$	–	–	$Cert_{KC^{-1}}(A, KR)$	$Cert_{-K_{CA}}(+K_A)$	B
Certificate by	$Cert\{A, K_A^+\}^{[CA]}$	–	–	–	$CA\langle\langle A\rangle\rangle$	A

In the table, the rightmost column classifies the notation in these groups:

C: The notation is commonly used in textbooks and other publications

B: The notation (or similar) is found in textbooks and other publications

A: The notation is believed to be unique for Aspen (invented here)

B.2 Notes on the typesetting options

Colors

The LaTeX package `aspen` provides the option `color`:

```
\usepackage[color]{aspen}
```

The package provides different color profiles. The default color profile is called `aspen`. Other color profiles are loaded by assigning a color profile to the `color` option. The following statement will load the same default color profile as the example above:

```
\usepackage[color=aspen]{aspen}
```

In addition, a few color profiles from Pygments are available: `autumn`, `colorful`, `default` (the default profile of Pygments), `emacs`, `friendly`, `gruvboxlight` (called `gruvbox-light` in Pygments), `manni`, and `staroffice`. Figure 6 shows the colors of all the color profiles of the Aspen package.

Other typesetting options

The default way of typesetting the public and the private key of the public-private key pair of A in Aspen is with a + superscript and a − superscript, like K_A^+ and K_A^- respectively. This behavior can be changed with the to package options `tradpubkey` and `tradprivkey`:

```
\usepackage[tradpubkey,tradprivkey]{aspen}
```

The result is that the public key of A will be typeset K_A and the private key of A will be typeset K_A^{-1}.

The default way of typesetting concatenation in Aspen is with the binary operator " . " (used to typeset concatenation of two values or strings). The Aspen package provides three options for typesetting concatenation: " . ", " || ", or " + ". This can be changed by passing a value to the `concat` option of the package. The valid values are `dot`, `dblbar`, and `plus`. The default is " . ". In this example " || " is chosen to be the concatenation operator:

```
\usepackage[concat=dblbar]{aspen}
```

aspen

Value	no	1, *true*
Principle	na	*A*
Key	kt	K_A
Nonce	nn	N_1
Timestamp	nt	T_s
String	sc	"Hello"
Variable	nv	x, y, z
Function	nf	$H(m)$
Procedure	err	*Abort*()
Code	go	\AKey{A}
Label	nl	M_1

autumn

Value	no	1, *true*
Principle	na	*A*
Key	kt	K_A
Nonce	nn	N_1
Timestamp	nt	T_s
String	sc	"Hello"
Variable	nv	x, y, z
Function	nf	$H(m)$
Procedure	err	*Abort*()
Code	go	\AKey{A}
Label	nl	M_1

colorful

Value	no	1, *true*
Principle	na	*A*
Key	kt	K_A
Nonce	nn	N_1
Timestamp	nt	T_s
String	sc	"Hello"
Variable	nv	x, y, z
Function	nf	$H(m)$
Procedure	err	*Abort*()
Code	go	\AKey{A}
Label	nl	M_1

default

Value	no	1, *true*
Principle	na	*A*
Key	kt	K_A
Nonce	nn	N_1
Timestamp	nt	T_s
String	sc	"Hello"
Variable	nv	x, y, z
Function	nf	$H(m)$
Procedure	err	*Abort*()
Code	go	\AKey{A}
Label	nl	M_1

emacs

Value	no	1, *true*
Principle	na	*A*
Key	kt	K_A
Nonce	nn	N_1
Timestamp	nt	T_s
String	sc	"Hello"
Variable	nv	x, y, z
Function	nf	$H(m)$
Procedure	err	*Abort*()
Code	go	\AKey{A}
Label	nl	M_1

friendly

Value	no	1, *true*
Principle	na	*A*
Key	kt	K_A
Nonce	nn	N_1
Timestamp	nt	T_s
String	sc	"Hello"
Variable	nv	x, y, z
Function	nf	$H(m)$
Procedure	err	*Abort*()
Code	go	\AKey{A}
Label	nl	M_1

gruvboxlight

Value	no	1, *true*
Principle	na	*A*
Key	kt	K_A
Nonce	nn	N_1
Timestamp	nt	T_s
String	sc	"Hello"
Variable	nv	x, y, z
Function	nf	$H(m)$
Procedure	err	*Abort*()
Code	go	\AKey{A}
Label	nl	M_1

manni

Value	no	1, *true*
Principle	na	*A*
Key	kt	K_A
Nonce	nn	N_1
Timestamp	nt	T_s
String	sc	"Hello"
Variable	nv	x, y, z
Function	nf	$H(m)$
Procedure	err	*Abort*()
Code	go	\AKey{A}
Label	nl	M_1

staroffice

Value	no	1, *true*
Principle	na	*A*
Key	kt	K_A
Nonce	nn	N_1
Timestamp	nt	T_s
String	sc	"Hello"
Variable	nv	x, y, z
Function	nf	$H(m)$
Procedure	err	*Abort*()
Code	go	\AKey{A}
Label	nl	M_1

Figure 6: The color profiles of the aspen package

B.3 Notation example listing

In this Section, all notation examples with their LaTeX code is listed without comments and any explanation.

B.3.1 Original Needham–Schroeder protocol

```
\begin{ASteps}
  \ASend{A}{S}{\APri{A}, \APri{B}, \ANonce{A}}[ons:1] \\
  \ASend*{S}{A}{\AEncrypted[big]{A,S}{\ANonce{A}, \APri{B},
      \AKey{A,B}, \AEncrypted{B,S}{\AKey{A,B}, \APri{A}}}}[ons:2] \\
  \ASend*{A}{B}{\AEncrypted{B,S}{\AKey{A,B}, \APri{A}}}[ons:3] \\
  \ASend*{B}{A}{\AEncrypted{A,B}{\ANonce{B}}}[ons:4] \\
  \ASend*{A}{B}{\AEncrypted{A,B}{\ANonce{B} - 1}}[ons:5]
\end{ASteps}
```

$M_1 \quad A \longrightarrow S : \{A, B, N'_A\}$

$M_2 \quad S \longrightarrow A : \left\{N'_A, B, K_{A,B}, \{K_{A,B}, A\}_{K_{B,S}}\right\}_{K_{A,S}}$

$M_3 \quad A \longrightarrow B : \{K_{A,B}, A\}_{K_{B,S}}$

$M_4 \quad B \longrightarrow A : \{N'_B\}_{K_{A,B}}$

$M_5 \quad A \longrightarrow B : \{N'_B - 1\}_{K_{A,B}}$

B.3.2 Revised Needham–Schroeder protocol

```
\begin{ASteps}
  \ASend{A}{B}{\APri{A}}[rns:1] \\
  \ASend*{B}{A}{\AEncrypted{B,S}{\APri{A},\ANonce{I}}}[rns:2] \\
  \ASend[big]{A}{S}{\APri{A}, \APri{B}, \ANonce{A},
    \AEncrypted{B,S}{\APri{A},\ANonce{I}}}[rns:3] \\
  \ASend*{S}{A}{\AEncrypted[big]{A,S}{\ANonce{A}, \APri{B},
      \AKey{A,B}, \AEncrypted{B,S}{\AKey{A,B}, \APri{A}, \ANonce{I}}}}[rns:4] \\
  \ASend*{A}{B}{\AEncrypted{B,S}{\AKey{A,B}, \APri{A}, \ANonce{I}}}[rns:5] \\
  \ASend*{B}{A}{\AEncrypted{A,B}{\ANonce{B}}}[rns:6] \\
  \ASend*{A}{B}{\AEncrypted{A,B}{\ANonce{B} - 1}}[rns:7]
\end{ASteps}
```

$M_1 \quad A \longrightarrow B : \{A\}$

$M_2 \quad B \longrightarrow A : \{A, N'_I\}_{K_{B,S}}$

$M_3 \quad A \longrightarrow S : \left\{A, B, N'_A, \{A, N'_I\}_{K_{B,S}}\right\}$

$M_4 \quad S \longrightarrow A : \left\{N'_A, B, K_{A,B}, \{K_{A,B}, A, N'_I\}_{K_{B,S}}\right\}_{K_{A,S}}$

$M_5 \quad A \longrightarrow B : \{K_{A,B}, A, N'_I\}_{K_{B,S}}$

$M_6 \quad B \longrightarrow A : \{N'_B\}_{K_{A,B}}$

$M_7 \quad A \longrightarrow B : \{N'_B - 1\}_{K_{A,B}}$

B.3.3 Otway-Rees protocol

```
\begin{ASteps}
  \ASend[big]{A}{B}{\ACounter{A}, \APri{A}, \APri{B},
    \AEncrypted{A,S}{\ANonce{A}, \ACounter{A}, \APri{A}, \APri{B}}}[or:1] \\
  \ASend[big]{B}{S}{\ACounter{A}, \APri{A}, \APri{B},
    \AEncrypted{A,S}{\ANonce{A}, \ACounter{A}, \APri{A}, \APri{B}},
    \AEncrypted{B,S}{\ANonce{B}, \ACounter{A}, \APri{A}, \APri{B}}}[or:2] \\
  \ASend[big]{S}{B}{\ACounter{A},
    \AEncrypted{A,S}{\ANonce{A}, \AKey'{A,B}},
    \AEncrypted{B,S}{\ANonce{B}, \AKey'{A,B}}}[or:3] \\
  \ASend[big]{B}{A}{\ACounter{A},
    \AEncrypted{A,S}{\ANonce{A}, \AKey'{A,B}}}[or:4]
\end{ASteps}
```

$M_1 \quad A \longrightarrow B : \{I_A, A, B, \{N'_A, I_A, A, B\}_{K_{A,S}}\}$

$M_2 \quad B \longrightarrow S : \{I_A, A, B, \{N'_A, I_A, A, B\}_{K_{A,S}}, \{N'_B, I_A, A, B\}_{K_{B,S}}\}$

$M_3 \quad S \longrightarrow B : \{I_A, \{N'_A, K'_{A,B}\}_{K_{A,S}}, \{N'_B, K'_{A,B}\}_{K_{B,S}}\}$

$M_4 \quad B \longrightarrow A : \{I_A, \{N'_A, K'_{A,B}\}_{K_{A,S}}\}$

B.3.4 Kerberos protocol

```
\begin{ASteps}
  \ASend{A}{S}{\APri{A}, \APri{B}} \\
  \ASend*{S}{A}{\AEncrypted[big]{A,S}{\ATS{s}, \ATTL{}, \AKey{A,B},
      \APri{B}, \AEncrypted{B,S}{\ATS{s}, \ATTL{}, \AKey{A,B}, \APri{A}}}} \\
  \ASend[big]{A}{B}{\AEncrypted{B,S}{\ATS{s}, \ATTL{}, \AKey{A,B},
      \APri{A}}, \AEncrypted{A,B}{\APri{A}, \ATS{a}}} \\
  \ASend*{B}{A}{\AEncrypted{A,B}{\ATS{a} + 1}}
\end{ASteps}
```

$M_1 \quad A \longrightarrow S : \{A, B\}$

$M_2 \quad S \longrightarrow A : \{T_s, L, K_{A,B}, B, \{T_s, L, K_{A,B}, A\}_{K_{B,S}}\}_{K_{A,S}}$

$M_3 \quad A \longrightarrow B : \{\{T_s, L, K_{A,B}, A\}_{K_{B,S}}, \{A, T_a\}_{K_{A,B}}\}$

$M_4 \quad B \longrightarrow A : \{T_a + 1\}_{K_{A,B}}$

B.3.5 Diffie–Hellman key exchange

```
\begin{ASteps}
  \ASend{A}{B}{\AVal{p}}[dh-ex:1] \\
  \AStepat*{A}{\AAssign{\AKey+{A}}{\ADhpubkeyf{A}(p)}}[dh-ex:2] \\
  \AStepat*{B}{\AAssign{\AKey+{B}}{\ADhpubkeyf{B}(p)}}[dh-ex:3] \\
  \ASend{A}{B}{\AKey+{A}}[dh-ex:4] \\
  \ASend{B}{A}{\AKey+{B}}[dh-ex:5] \\
  \AStepat*{A}{\AAssign{\AKey{A,B}}{\ADhkeyf{A}{B}(p)}}[dh-ex:6] \\
  \AStepat*{B}{\AAssign{\AKey{A,B}}{\ADhkeyf{B}{A}(p)}}[dh-ex:7]
\end{ASteps}
```

M_1 $A \longrightarrow B : \{p\}$

S_1 $A : K_A^+ \leftarrow DHPubKey(K_A^-, p)$

S_2 $B : K_B^+ \leftarrow DHPubKey(K_B^-, p)$

M_2 $A \longrightarrow B : \{K_A^+\}$

M_3 $B \longrightarrow A : \{K_B^+\}$

S_3 $A : K_{A,B} \leftarrow DHKey(K_A^-, K_B^+, p)$

S_4 $B : K_{A,B} \leftarrow DHKey(K_B^-, K_A^+, p)$

B.3.6 Needham–Schroeder public key protocol

```
\begin{ASteps}
  \ASend{A}{S}{\APri{A}, \APri{B}}[ns-pk:1] \\
  \ASend*{S}{A}{\ASigned-{S}{\AKey+{B},\APri{B}}}[ns-pk:2] \\
  \ASend*{A}{B}{\AEncrypted+{B}{\ANonce{A},\APri{A}}}[ns-pk:3] \\
  \ASend{B}{S}{\APri{B}, \APri{A}}[ns-pk:4] \\
  \ASend*{S}{B}{\ASigned-{S}{\AKey+{A},\APri{A}}}[ns-pk:5] \\
  \ASend*{B}{A}{\AEncrypted+{A}{\ANonce{A},\ANonce{B}}}[ns-pk:6] \\
  \ASend*{A}{B}{\AEncrypted+{B}{\ANonce{B}}}[ns-pk:7]
\end{ASteps}
```

M_1 $A \longrightarrow S : \{A, B\}$

M_2 $S \longrightarrow A : \{K_B^+, B\}^{K_S^-}$

M_3 $A \longrightarrow B : \{N_A', A\}_{K_B^+}$

M_4 $B \longrightarrow S : \{B, A\}$

M_5 $S \longrightarrow B : \{K_A^+, A\}^{K_S^-}$

M_6 $B \longrightarrow A : \{N_A', N_B'\}_{K_A^+}$

M_7 $A \longrightarrow B : \{N_B'\}_{K_B^+}$

B.3.7 Needham–Schroeder-Lowe public key protocol

```
\begin{ASteps}
  \ASend{A}{S}{\APri{A}, \APri{B}}[nsl-pk:1] \\
  \ASend*{S}{A}{\ASigned-{S}{\AKey+{B},\APri{B}}}[nsl-pk:2] \\
  \ASend*{A}{B}{\AEncrypted+{B}{\ANonce{A},\APri{A}}}[nsl-pk:3] \\
  \ASend{B}{S}{\APri{B}, \APri{A}}[nsl-pk:4] \\
  \ASend*{S}{B}{\ASigned-{S}{\AKey+{A},\APri{A}}}[nsl-pk:5] \\
  \ASend*{B}{A}{\AEncrypted+{A}{\ANonce{A},\ANonce{B},\APri{B}}}[nsl-pk:6] \\
  \ASend*{A}{B}{\AEncrypted+{B}{\ANonce{B}}}[nsl-pk:7]
\end{ASteps}
```

M_1 $A \longrightarrow S : \{A, B\}$

M_2 $S \longrightarrow A : \{K_B^+, B\}^{K_S^-}$

M_3 $A \longrightarrow B : \{N_A', A\}_{K_B^+}$

M_4 $B \longrightarrow S : \{B, A\}$

M_5 $S \longrightarrow B : \{K_A^+, A\}^{K_S^-}$

M_6 $B \longrightarrow A : \{N_A', N_B', B\}_{K_A^+}$

M_7 $A \longrightarrow B : \{N_B'\}_{K_B^+}$

B.3.8 ASW exchange protocol

```
\begin{ASteps}
  \ASend*{O}{R}{\ASigned-[big]{O}{%
      \AKey+{A},\AKey+{B},m,\AChash{\ANonce{O}}}}[asW:1] \\
  \ASend*{R}{O}{\ASigned-[Big]{R}{%
      \ASigned-[big]{O}{\AKey+{A},\AKey+{B},m,\AChash{\ANonce{O}}},
      \AChash{\ANonce{R}}}}[asw:2] \\
  \ASend{O}{R}{\ANonce{O}}[asw:3] \\
  \ASend{R}{O}{\ANonce{R}}[asw:4]
\end{ASteps}
```

M_1 $O \longrightarrow R : \left\{K_A^+, K_B^+, m, H\{N_O'\}\right\}^{K_O^-}$

M_2 $R \longrightarrow O : \left\{\left\{K_A^+, K_B^+, m, H\{N_O'\}\right\}^{K_O^-}, H\{N_R'\}\right\}^{K_R^-}$

M_3 $O \longrightarrow R : \{N_O'\}$

M_4 $R \longrightarrow O : \{N_R'\}$

B.3.9 Wide-mouthed-frog protocol

```
\begin{ASteps}
  \ASend[big]{A}{S}{A, \AEncrypted{A,S}{\ATS{A}, \APri{B}, \AKey'{A,B}}}[wmf:1] \\
  \ASend*[big]{S}{B}{\AEncrypted{B,S}{\ATS{S}, \APri{A}, \AKey'{A,B}}}[wmf:2]
\end{ASteps}
```

M_1 $A \longrightarrow S : \left\{A, \{T_A, B, K_{A,B}'\}_{K_{A,S}}\right\}$

M_2 $S \longrightarrow B : \{T_S, A, K_{A,B}'\}_{K_{B,S}}$

B.3.10 Idealized wide-mouthed-frog protocol

```
\begin{ASteps}
  \ASend*{A}{S}{\AEncrypted[big]{A,S}{\ATS{A},
      \{\APri{A}\BANSharedkey{\AKey{A,B}}\APri{B}\}}}[iwmf:1] \\
  \ASend*[big]{S}{B}{\AEncrypted{B,S}{\ATS{S},
      \APri{A}\BANBelieves\APri{A}\BANSharedkey{\AKey{A,B}}\APri{B}}}[iwmf:2]
\end{ASteps}
```

M_1 $A \longrightarrow S : \left\{T_A, \{A \overset{K_{A,B}}{\longleftrightarrow} B\}\right\}_{K_{A,S}}$

M_2 $S \longrightarrow B : \{T_S, A \mid\equiv A \overset{K_{A,B}}{\longleftrightarrow} B\}_{K_{B,S}}$

B.3.11 Wide-mouthed-frog protocol with declarations

```
\begin{ASteps}[labels=D]
  \AStep(D){\APri{A}\BANSharedkey{\AKey{A,S}}\APri{S};
    \APri{B}\BANSharedkey{\AKey{B,S}}\APri{S}}[dwmf:1] \\
  \AStep(D){\APri{A}\BANSees\AKey'{A,B}; \BANFresh{\AKey'{A,B}};
    \APri{A}\BANSees\AVal{m}}[dwmf:3] \\
  \ASend[big]{A}{S}{A,\AEncrypted{A,S}{\ATS{A},\APri{B},\AKey'{A,B}}}[dwmf:4] \\
  \ASend*[big]{S}{B}{\AEncrypted{B,S}{\ATS{S},\APri{A},\AKey'{A,B}}}[dwmf:5]
\end{ASteps}
```

D_1 $A \overset{K_{A,S}}{\longleftrightarrow} S$; $B \overset{K_{B,S}}{\longleftrightarrow} S$

D_2 $A \triangleleft K'_{A,B}$; $\sharp(K'_{A,B})$; $A \triangleleft m$

M_1 $A \longrightarrow S : \{A, \{T_A, B, K'_{A,B}\}_{K_{A,S}}\}$

M_2 $S \longrightarrow B : \{T_S, A, K'_{A,B}\}_{K_{B,S}}$

B.3.12 TLS 1.3 Handshake

```
\begin{ASteps}
  \AStepat*{C}{\AAssign{\AKey+{C_e}}{\ADhpubkeyf{C_e}},
    \BANFresh{\AKey-{C_e}}}[tls-hs:1] \\
  \ASend{C}{S}{\ldots, \ANonce{C}, \AKey+{C_e}, \ldots}[tls-hs:2] \\
  \AStepat*{S}{\AAssign{\AKey+{S_e}}{\ADhpubkeyf{S_e}},
    \BANFresh{\AKey-{S_e}}}[tls-hs:3] \\
  \ASend[big]{S}{C}{\ldots, \ANonce{S}, \AKey+{S_e}, \ACert-{CA}{S},
    \ASigned-{S}{\ANonce{C'}, \ANonce{S'}, \ldots}, \ldots}[tls-hs:4] \\
  \AStepat*{C}{\ACond[\AVerify+[big]{CA}{\ACert-{CA}{S}}]{%
      \ACondstep{\AFalse}{\AAbort}}}[tls-hs:5a] \\
  \AStepat*{C}{\ACond[\AVerify+{S}{\ASigned-{S}{\ANonce{C}, \ANonce{S},
        \ldots}}]{\ACondstep{\AFalse}{\AAbort}}}[tls-hs:5b] \\
  \AStepat*{C}{\ACond[\ANonce{C}=\ANonce{C'}\land\ANonce{S}=\ANonce{S'}]{%
      \ACondstep{\AFalse}{\AAbort}}}[tls-hs:5c] \\
  \AStepat*{C}{\AAssign{\AKey{C,S}}{\ADhkeyf{C_e}{S_e}}}[tls-hs:6a] \\
  \ASend*{C}{S}{\AEncrypted{C,S}{\ldots}}[tls-hs:7] \\
  \AStepat*{S}{\AAssign{\AKey{C,S}}{\ADhkeyf{S_e}{C_e}}}[tls-hs:6b]
\end{ASteps}
```

S_1 $C : K^+_{C_e} \leftarrow DHPubKey(K^-_{C_e}), \sharp(K^-_{C_e})$

M_1 $C \longrightarrow S : \{\ldots, N'_C, K^+_{C_e}, \ldots\}$

S_2 $S : K^+_{S_e} \leftarrow DHPubKey(K^-_{S_e}), \sharp(K^-_{S_e})$

M_2 $S \longrightarrow C : \{\ldots, N'_S, K^+_{S_e}, Cert\{S, K^+_S\}^{K^-_{CA}}, \{N'_{C'}, N'_{S'}, \ldots\}^{K^-_S}, \ldots\}$

S_3 $C : Verify\left(K^+_{CA}, Cert\{S, K^+_S\}^{K^-_{CA}}\right)$? $false$: $\underline{Abort}()$

S_4 $C : Verify(K^+_S, \{N'_C, N'_S, \ldots\}^{K^-_S})$? $false$: $\underline{Abort}()$

S_5 $C : N'_C = N'_{C'} \land N'_S = N'_{S'}$? $false$: $\underline{Abort}()$

S_6 $C : K_{C,S} \leftarrow DHKey(K^-_{C_e}, K^+_{S_e})$

M_3 $C \longrightarrow S : \{\ldots\}_{K_{C,S}}$

S_7 $S : K_{C,S} \leftarrow DHKey(K^-_{S_e}, K^+_{C_e})$

B.3.13 SMC: Calculate the mean value

```
\begin{ASteps}\setcounter{counterS}{-1}%
  \AStepat*{C}{\AAssign{\AVal{V'_0}}{\ARandom'{0}}}\\
  \ASend*{C}{P_1}{\ASigned-[Big]{C}{%
      \AEncrypted+[big]{P_1}{\ASigned-{C}{\AVal{V_0}}},
      \AEncrypted+[big]{P_1}{\ASigned-{C}{\APri{C},\APri{P_2}}},
      \AEncrypted+[big]{P_2}{\ASigned-{C}{\APri{P_1},\APri{P_3}}},
      \AEncrypted+[big]{P_3}{\ASigned-{C}{\APri{P_2},\APri{C}}},
      \AEncrypted+[big]{C}{\ASigned-{C}{\APri{C},\_\,}}}} \\
  \AStepat*{P_1}{\ACond[\AVerify+[big]{C}{\ASigned-{C}{\APri{C},\APri{P_2}}}%
    \land \AVerify+[big]{C}{\ASigned-{C}{\ldots}}]{%
      \ACondstep{\AFalse}{\AAbort}}} \\
  \AStepat*{P_1}{\ACond[\AVerify+[big]{C}{\ASigned-{C}{\AVal{V'_0}}}]{%
      \ACondstep{\AFalse}{\AAbort}\ANextbreak
      \ACondstep{\ATrue}{\AAssign{\AVal{V'_1}}{\AVal{V'_0}+\AVal{V_1}}}}} \\
  \ASend*{P_1}{P_2}{\ASigned-[Big]{P_1}{%
      \AEncrypted+[big]{P_2}{\ASigned-{P_1}{\AVal{V'_1}}},
      \AEncrypted+[big]{P_2}{\ASigned-{C}{\APri{P_1},\APri{P_3}}},
      \AEncrypted+[big]{P_3}{\ASigned-{C}{\APri{P_2},\APri{C}}},
      \AEncrypted+[big]{C}{\ASigned-{C}{\APri{C},\_\,}}} \\
  \AStepat*{P_2}{\ACond[\AVerify+[big]{C}{\ASigned-{C}{\APri{P_1},\APri{P_3}}}%
    \land \AVerify+[big]{P_1}{\ASigned-{P_1}{\ldots}}]{%
      \ACondstep{\AFalse}{\AAbort}}} \\
  \AStepat*{P_2}{\ACond[\AVerify+[big]{P_1}{\ASigned-{P_1}{\AVal{V'_1}}}]{%
      \ACondstep{\AFalse}{\AAbort}\ANextbreak
      \ACondstep{\ATrue}{\AAssign{\AVal{V'_2}}{\AVal{V'_1}+\AVal{V_2}}}} \\
  \ASend*{P_2}{P_3}{\ASigned-[Big]{P_2}{%
      \AEncrypted+[big]{P_3}{\ASigned-{P_2}{\AVal{V'_2}}},
      \AEncrypted+[big]{P_3}{\ASigned-{C}{\APri{P_2},\APri{C}}},
      \AEncrypted+[big]{C}{\ASigned-{C}{\APri{C},\_\,}}} \\
  \AStepat*{P_3}{\ACond[\AVerify+[big]{C}{\ASigned-{C}{\APri{P_2},\APri{C}}}%
    \land \AVerify+[big]{P_2}{\ASigned-{P_2}{\ldots}}]{%
      \ACondstep{\AFalse}{\AAbort}}} \\
  \AStepat*{P_3}{\ACond[\AVerify+[big]{P_2}{\ASigned-{P_2}{\AVal{V'_2}}}]{%
      \ACondstep{\AFalse}{\AAbort}\ANextbreak
      \ACondstep{\ATrue}{\AAssign{\AVal{V'_3}}{\AVal{V'_2}+\AVal{V_3}}}} \\
  \ASend*{P_3}{C}{\ASigned-[Big]{P_3}{%
      \AEncrypted+[big]{C}{\ASigned-{P_3}{\AVal{V'_3}}},
      \AEncrypted+[big]{C}{\ASigned-{C}{\APri{C},\_\,}}} \\
  \AStepat*{C}{\ACond[\AVerify+[big]{C}{\ASigned-{C}{\APri{C},\_\,}}%
    \land \AVerify+[big]{P_3}{\ASigned-{P_3}{\ldots}}]{%
      \ACondstep{\AFalse}{\AAbort}}} \\
  \AStepat*{C}{\ACond[\AVerify+[big]{P_3}{\ASigned-{P_3}{\AVal{V'_3}}}]{%
      \ACondstep{\AFalse}{\AAbort}\ANextbreak
      \ACondstep{\ATrue}{\AAssign{\AVal{M}}{(\AVal{V'_3}-\AVal{V'_0})/\AVal{3}}}}}
\end{ASteps}
```

$$
\begin{array}{lll}
S_0 & & C : V_0' \leftarrow R_0' \\
M_1 & C \longrightarrow P_1 & : \left\{\left\{\{V_0\}^{K_C^-}\right\}_{K_{P_1}^+}, \left\{\{C, P_2\}^{K_C^-}\right\}_{K_{P_1}^+}, \left\{\{P_1, P_3\}^{K_C^-}\right\}_{K_{P_2}^+}, \left\{\{P_2, C\}^{K_C^-}\right\}_{K_{P_3}^+}, \left\{\{C, _\}^{K_C^-}\right\}_{K_C^+}\right\}^{K_C^-} \\
S_1 & & P_1 : \mathit{Verify}\left(K_C^+, \{C, P_2\}^{K_C^-}\right) \wedge \mathit{Verify}\left(K_C^+, \{\ldots\}^{K_C^-}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \\
S_2 & & P_1 : \mathit{Verify}\left(K_C^+, \{V_0'\}^{K_C^-}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \ ;;\ \mathit{true} : V_1' \leftarrow V_0' + V_1 \\
M_2 & P_1 \longrightarrow P_2 & : \left\{\left\{\{V_1'\}^{K_{P_1}^-}\right\}_{K_{P_2}^+}, \left\{\{P_1, P_3\}^{K_C^-}\right\}_{K_{P_2}^+}, \left\{\{P_2, C\}^{K_C^-}\right\}_{K_{P_3}^+}, \left\{\{C, _\}^{K_C^-}\right\}_{K_C^+}\right\}^{K_{P_1}^-} \\
S_3 & & P_2 : \mathit{Verify}\left(K_C^+, \{P_1, P_3\}^{K_C^-}\right) \wedge \mathit{Verify}\left(K_{P_1}^+, \{\ldots\}^{K_{P_1}^-}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \\
S_4 & & P_2 : \mathit{Verify}\left(K_{P_1}^+, \{V_1'\}^{K_{P_1}^-}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \ ;;\ \mathit{true} : V_2' \leftarrow V_1' + V_2 \\
M_3 & P_2 \longrightarrow P_3 & : \left\{\left\{\{V_2'\}^{K_{P_2}^-}\right\}_{K_{P_3}^+}, \left\{\{P_2, C\}^{K_C^-}\right\}_{K_{P_3}^+}, \left\{\{C, _\}^{K_C^-}\right\}_{K_C^+}\right\}^{K_{P_2}^-} \\
S_5 & & P_3 : \mathit{Verify}\left(K_C^+, \{P_2, C\}^{K_C^-}\right) \wedge \mathit{Verify}\left(K_{P_2}^+, \{\ldots\}^{K_{P_2}^-}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \\
S_6 & & P_3 : \mathit{Verify}\left(K_{P_2}^+, \{V_2'\}^{K_{P_2}^-}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \ ;;\ \mathit{true} : V_3' \leftarrow V_2' + V_3 \\
M_4 & P_3 \longrightarrow C & : \left\{\left\{\{V_3'\}^{K_{P_3}^-}\right\}_{K_C^+}, \left\{\{C, _\}^{K_C^-}\right\}_{K_C^+}\right\}^{K_{P_3}^-} \\
S_7 & & C : \mathit{Verify}\left(K_C^+, \{C, _\}^{K_C^-}\right) \wedge \mathit{Verify}\left(K_{P_3}^+, \{\ldots\}^{K_{P_3}^-}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \\
S_8 & & C : \mathit{Verify}\left(K_{P_3}^+, \{V_3'\}^{K_{P_3}^-}\right) \ ?\ \mathit{false} : \underline{\mathit{Abort}}() \ ;;\ \mathit{true} : M \leftarrow (V_3' - V_0')/3
\end{array}
$$

B.3.14 File sharing with symmetric and public key encryption

```
\begin{ASteps}
  \AStepat*{A}{\BANFresh{\AKey{F_1}}, \BANFresh{\AKey{F_2}}}[fs-pub:1] \\
  \AStepat*{A}{%
    \AEncrypt{F_1}{F_1}[*],
    \AEncrypt{F_2}{F_2}[*]}[fs-pub:2] \\
  \AStepat*{A}{%
    \AEncrypt+{A}{\AKey{F_1}}[*],
    \AEncrypt+{B}{\AKey{F_1}}[*]}[fs-pub:3] \\
  \AStepat*{A}{\AEncrypt+{A}{\AKey{F_2}}[*]}[fs-pub:4] \\
  \ASend[big]{A}{S}{\AEncrypted+{A}{\AKey{F_1}},
    \AEncrypted+{B}{\AKey{F_1}}, \AEncrypted{F_1}{F_1}}[fs-pub:5] \\
  \ASend[big]{A}{S}{\AEncrypted+{A}{\AKey{F_2}},
      \AEncrypted{F_2}{F_2}}[fs-pub:6] \\
  \ASend[big]{S}{B}{\AEncrypted+{A}{\AKey{F_1}},
    \AEncrypted+{B}{\AKey{F_1}}, \AEncrypted{F_1}{F_1}}[fs-pub:7] \\
  \AStepat*{B}{%
    \ADecrypt"{P_B}{\AEncrypted"{P_B}{\AKey-{B}}}[\AKey-{B}],
    \ADecrypt-{B}{\AEncrypted+{B}{\AKey{F_1}}}[\AKey{F_1}]}[fs-pub:8] \\
  \AStepat*{B}{\ADecrypt{F_1}{\AEncrypted{F_1}{F_1}}[\AVal{F_1}]}[fs-pub:9]
\end{ASteps}
```

S_1 $\quad A : \sharp(K_{F_1}), \sharp(K_{F_2})$

S_2 $\quad A : Encrypt(K_{F_1}, F_1) \rightarrow \{F_1\}_{K_{F_1}}, Encrypt(K_{F_2}, F_2) \rightarrow \{F_2\}_{K_{F_2}}$

S_3 $\quad A : Encrypt(K_A^+, K_{F_1}) \rightarrow \{K_{F_1}\}_{K_A^+}, Encrypt(K_B^+, K_{F_1}) \rightarrow \{K_{F_1}\}_{K_B^+}$

S_4 $\quad A : Encrypt(K_A^+, K_{F_2}) \rightarrow \{K_{F_2}\}_{K_A^+}$

M_1 $\quad A \longrightarrow S : \left\{\{K_{F_1}\}_{K_A^+}, \{K_{F_1}\}_{K_B^+}, \{F_1\}_{K_{F_1}}\right\}$

M_2 $\quad A \longrightarrow S : \left\{\{K_{F_2}\}_{K_A^+}, \{F_2\}_{K_{F_2}}\right\}$

M_3 $\quad S \longrightarrow B : \left\{\{K_{F_1}\}_{K_A^+}, \{K_{F_1}\}_{K_B^+}, \{F_1\}_{K_{F_1}}\right\}$

S_5 $\quad B : Decrypt(K''_{P_B}, \{K_B^-\}_{K''_{P_B}}) \rightarrow K_B^-, Decrypt(K_B^-, \{K_{F_1}\}_{K_B^+}) \rightarrow K_{F_1}$

S_6 $\quad B : Decrypt(K_{F_1}, \{F_1\}_{K_{F_1}}) \rightarrow F_1$

B.3.15 Scalable secure file sharing

```
\begin{ASteps}
  \AStepat*{A}{%
    \BANFresh{\AKey{F_1}}; \BANFresh{\AKey{F_2}};
    \BANFresh{\AKey{S_A}}; \BANFresh{\AKey{S_{A,B}}};
    \BANFresh{\AKey-{S_A}}; \BANFresh{\AKey+{S_A}};
    \BANFresh{\AKey-{S_{A,B}}}; \BANFresh{\AKey+{S_{A,B}}}} \\
  \AStepat*{A}{%
    \AEncrypt{F_1}{F_1}[*]; \AEncrypt{F_2}{F_2}[*]} \\
  \AStepat*{A}{%
    \AEncrypt+{S_{A,B}}{\AKey{F_1}}[*]; \AEncrypt+{S_A}{\AKey{F_2}}[*]} \\
  \AStepat*{A}{%
    \AEncrypt{S_A}{\AKey-{S_A}}[*]; \AEncrypt{S_{A,B}}{\AKey-{S_{A,B}}}[*]} \\
  \AStepat*{A}{%
    \AEncrypt+{A}{\AKey{S_A}}[*]} \\
  \AStepat*{A}{%
    \AEncrypt+{A}{\AKey{S_{A,B}}}[*]; \AEncrypt+{B}{\AKey{S_{A,B}}}[*]} \\
  \ASend[big]{A}{S}{%
    \AEncrypted+{A}{\AKey{S_A}}, \AEncrypted+{A}{\AKey{S_{A,B}}},
    \AEncrypted+{B}{\AKey{S_{A,B}}}} \\
  \ASend[big]{A}{S}{%
    \AEncrypted{S_A}{\AKey-{S_A}}, \AEncrypted{S_{A,B}}{\AKey-{S_{A,B}}}} \\
  \ASend[big]{A}{S}{%
    \AEncrypted+{S_{A,B}}{\AKey{F_1}}, \AEncrypted{F_1}{F_1}} \\
  \ASend[big]{A}{S}{\AEncrypted+{S_A}{\AKey{F_2}}, \AEncrypted{F_2}{F_2}} \\
  \ASend[big]{S}{B}{\AEncrypted+{B}{\AKey{S_{A,B}}},
    \AEncrypted{S_{A,B}}{\AKey-{S_{A,B}}}} \\
  \ASend[big]{S}{B}{%
    \AEncrypted+{S_{A,B}}{\AKey{F_1}}, \AEncrypted{F_1}{F_1}} \\
  \AStepat*{B}{%
    \ADecrypt"{P_B}{\AEncrypted"{P_B}{\AKey-{B}}}[\AKey-{B}];
    \ADecrypt-{B}{\AEncrypted+{B}{\AKey{S_{A,B}}}}[\AKey{S_{A,B}}]} \\
  \AStepat*{B}{%
    \ADecrypt{S_{A,B}}{\AEncrypted{S_{A,B}}{\AKey-{S_{A,B}}}}[\AKey-{S_{A,B}}];
    \ADecrypt-{S_{A,B}}{\AEncrypted+{S_{A,B}}{\AKey{F_1}}}[\AKey{F_1}]} \\
  \AStepat*{B}{\ADecrypt{F_1}{\AEncrypted{F_1}{F_1}}[\AVal{F_1}]}
\end{ASteps}
```

S_1 $A : \sharp(K_{F_1}); \sharp(K_{F_2}); \sharp(K_{S_A}); \sharp(K_{S_{A,B}}); \sharp(K^-_{S_A}); \sharp(K^+_{S_A}); \sharp(K^-_{S_{A,B}}); \sharp(K^+_{S_{A,B}})$

S_2 $A : Encrypt(K_{F_1}, F_1) \rightarrow \{F_1\}_{K_{F_1}}; Encrypt(K_{F_2}, F_2) \rightarrow \{F_2\}_{K_{F_2}}$

S_3 $A : Encrypt(K^+_{S_{A,B}}, K_{F_1}) \rightarrow \{K_{F_1}\}_{K^+_{S_{A,B}}}; Encrypt(K^+_{S_A}, K_{F_2}) \rightarrow \{K_{F_2}\}_{K^+_{S_A}}$

S_4 $A : Encrypt(K_{S_A}, K^-_{S_A}) \rightarrow \{K^-_{S_A}\}_{K_{S_A}}; Encrypt(K_{S_{A,B}}, K^-_{S_{A,B}}) \rightarrow \{K^-_{S_{A,B}}\}_{K_{S_{A,B}}}$

S_5 $A : Encrypt(K^+_A, K_{S_A}) \rightarrow \{K_{S_A}\}_{K^+_A}$

S_6 $A : Encrypt(K^+_A, K_{S_{A,B}}) \rightarrow \{K_{S_{A,B}}\}_{K^+_A}; Encrypt(K^+_B, K_{S_{A,B}}) \rightarrow \{K_{S_{A,B}}\}_{K^+_B}$

M_1 $A \longrightarrow S : \left\{\{K_{S_A}\}_{K^+_A}, \{K_{S_{A,B}}\}_{K^+_A}, \{K_{S_{A,B}}\}_{K^+_B}\right\}$

M_2 $A \longrightarrow S : \left\{\{K^-_{S_A}\}_{K_{S_A}}, \{K^-_{S_{A,B}}\}_{K_{S_{A,B}}}\right\}$

M_3 $A \longrightarrow S : \left\{\{K_{F_1}\}_{K^+_{S_{A,B}}}, \{F_1\}_{K_{F_1}}\right\}$

M_4 $A \longrightarrow S : \left\{\{K_{F_2}\}_{K^+_{S_A}}, \{F_2\}_{K_{F_2}}\right\}$

M_5 $S \longrightarrow B : \left\{\{K_{S_{A,B}}\}_{K^+_B}, \{K^-_{S_{A,B}}\}_{K_{S_{A,B}}}\right\}$

M_6 $S \longrightarrow B : \left\{\{K_{F_1}\}_{K^+_{S_{A,B}}}, \{F_1\}_{K_{F_1}}\right\}$

S_7 $B : Decrypt(K^{''}_{P_B}, \{K^-_B\}_{K^{''}_{P_B}}) \rightarrow K^-_B; Decrypt(K^-_B, \{K_{S_{A,B}}\}_{K^+_B}) \rightarrow K_{S_{A,B}}$

S_8 $B : Decrypt(K_{S_{A,B}}, \{K^-_{S_{A,B}}\}_{K_{S_{A,B}}}) \rightarrow K^-_{S_{A,B}}; Decrypt(K^-_{S_{A,B}}, \{K_{F_1}\}_{K^+_{S_{A,B}}}) \rightarrow K_{F_1}$

S_9 $B : Decrypt(K_{F_1}, \{F_1\}_{K_{F_1}}) \rightarrow F_1$